Hostel Complaint & Attender Duty Tracking System

Phase 1 — Design Document

Ajoy Mathew (CS23B101) CS5013 — Programming with AI, IIT Madras

11 September 2026

Contents

1. Overview & Scope **3**
1.1 Problem . . . 3
1.2 Goals . . . 3
1.3 Non-goals . . . 4
1.4 Actors . . . 4
1.5 Glossary . . . 5
1.6 Non-functional requirements . . . 5
1.7 Open questions . . . 6
1.8 Constraints . . . 6

2. System Architecture **8**
2.1 Architectural style . . . 8
2.2 Context diagram . . . 8
2.3 Module breakdown . . . 8
2.4 Layering rules . . . 10
2.5 The pipeline . . . 10
2.6 Deployment topology . . . 11
2.7 Cross-cutting concerns . . . 12

3. Data Model **13**
3.1 Design principles . . . 13
3.2 Entity–relationship diagram . . . 13
3.3 Identity & access . . . 13
3.4 Locations, coverage & availability . . . 14
3.5 Ingestion & complaints . . . 14
3.6 Tasks, dockets and the ledger . . . 16
3.7 Audit and classifier data . . . 17
3.8 Retention . . . 18

4. State Machines **19**
4.1 Why this is formalised . . . 19
4.2 Complaint lifecycle . . . 19
4.3 The CMS docket . . . 21
4.4 Task lifecycle (special duties) . . . 22
4.5 Priority and aging . . . 22

5. Ingestion & Categorization **23**
5.1 The hard part . . . 23
5.2 Column mapping . . . 23
5.3 Delta detection . . . 23

5.4 Normalization to a complaint . . . 25
5.5 Categorization . . . 25
5.6 Meeting the 90% accuracy criterion . . . 26
5.7 OpenNLP as a deferred second stage . . . 26
5.8 CSV fallback . . . 26

6. Dispatch Heuristic **27**
6.1 Problem statement . . . 27
6.2 Formulation . . . 27
6.3 Shared tasks . . . 29
6.4 Degenerate cases . . . 29
6.5 Why greedy, and not optimal assignment . . . 29

7. Points Ledger & Concurrency Control **30**
7.1 What can actually go wrong . . . 30
7.2 Mechanisms . . . 30
7.3 Why double-crediting is structurally impossible . . . 30
7.4 Idempotency at the HTTP boundary . . . 31
7.5 Shared tasks . . . 31
7.6 Claiming a duty (C4) . . . 32
7.7 Optimistic locking and conflict UX . . . 32
7.8 The leaderboard . . . 32
7.9 Month boundaries . . . 33

8. Analytics & Telemetry **34**
8.1 Where the numbers come from . . . 34
8.2 MTTR and bottleneck detection . . . 34
8.3 Anomaly detection . . . 35
8.4 Dashboard visualizations . . . 36
8.5 Explicit limitations . . . 36

9. API & Data Contracts **37**
9.1 Shape of the web layer . . . 37
9.2 Conventions . . . 37
9.3 Attender API (mobile) . . . 37
9.4 Manager API . . . 39
9.5 SSE contract . . . 39
9.6 Push notification contract . . . 40
9.7 Error model . . . 42

10. Frontend Design **43**
10.1 Two clients, two very different users . . . 43
10.2 Attender mobile . . . 43
10.3 Manager desktop . . . 44
10.4 Screens . . . 45
10.5 Styling & accessibility . . . 46
10.6 Performance . . . 46

11. Security & Privacy **47**
11.1 Threat model . . . 47
11.2 Authentication . . . 47
11.3 Authorization . . . 48
11.4 PII and data minimisation . . . 49
11.5 Secrets . . . 49
11.6 Application security . . . 50
11.7 Audit as a security control . . . 50

12. Test Plan **51**
12.1 Strategy . . . 51
12.2 Fixtures . . . 51
12.3 Categorization gold set . . . 52
12.4 Corner-case suite . . . 52
12.5 Concurrency suite . . . 53
12.6 Acceptance-criteria traceability . . . 54
12.7 What we are not testing . . . 54

13. Delivery Plan **56**
13.1 Strategy: a working system first, capability layered on top . . . 56
13.2 Fixed dates . . . 56
13.3 Phase 1 — Walking skeleton · 12–21 Sep . . . 56
13.4 Phase 2 — Automate the daily work · 22 Sep – 5 Oct . . . 57
13.5 Phase 3 — Complete the points story · 10–20 Oct . . . 58
13.6 Phase 4 — Oversight and reach · 21–30 Oct . . . 58
13.7 Phase 5 — Hardening and sign-off · 31 Oct – 6 Nov . . . 59
13.8 Backlog — deliberately not scheduled . . . 59
13.9 Changes from the original proposal . . . 59
13.10 Risk and the cut line . . . 60

14. Decision Record **61**
ADR-001 · One PostgreSQL database; the standby runs against it too . . . 61
ADR-002 · Google sign-in for everyone, admitted by an allowlist . . . 61
ADR-003 · SSE, not WebSocket, for live updates . . . 62
ADR-004 · Rule-based lexicon as the primary classifier; OpenNLP deferred . . . 62
ADR-005 · Greedy per-complaint dispatch, not global optimisation . . . 63
ADR-006 · Pessimistic locking for duty claims, optimistic elsewhere . . . 63
ADR-007 · `READ COMMITTED` isolation, not `SERIALIZABLE` . . . 63
ADR-008 · Append-only ledger; no stored point balance . . . 64
ADR-009 · Content-derived identity for source rows, not row index . . . 64
ADR-010 · Modular monolith, not microservices . . . 64
ADR-011 · Web Push with VAPID for attender notifications, not FCM or a native app . . . 65
ADR-012 · Deliberate simplification pass; ship a working system first . . . 65

1. Overview & Scope

1.1 Problem

The Hostel Manager at Cauvery Hostel, IIT Madras processes daily maintenance complaints by hand. Residents submit a Google Form; the manager copies each response into a spreadsheet, sorts it into a domain (Electrical, Civil, RO Plant), checks which attenders are on duty and which floors they cover, assigns the work verbally, chases inspection IDs, and at month end tallies points for bonus calculation. This costs **30–45 minutes every day** and is error-prone at every step.

Four attenders serve the hostel. Points accrue at **+2** per resolved complaint, custom values for special duties, and **−1** per logged infraction. All of it is currently reconciled by hand in a spreadsheet that is easily corrupted when edited from mobile devices.

1.2 Goals

G	Goal	Measured by
G1	Eliminate manual copy-paste from the Form response sheet	Zero manual transcription; no duplicate complaints across polls

G	Goal	Measured by
G2	Auto-categorize complaints into the correct domain	≥90% accuracy on the labelled gold set (Sec. 12.3)
G3	Assign each complaint to the most appropriate attender automatically	Dispatch produces an assignment or an explicit triage reason, 100% of the time
G4	Give attenders a usable mobile queue	Attender can log a CMS docket and resolve a ticket in ≤3 taps from the queue
G5	Make point accounting correct and auditable	No double-credit under concurrent updates; every point movement traceable to an actor and a cause
G6	Replace the end-of-month manual tally	One-click `.xlsx` export; leaderboard always live
G7	Surface operational insight the manager cannot currently see	MTTR per category, bottleneck stage, anomaly alerts
G8	Reach an attender when work is assigned, without them opening the app	Push notification delivered to the attender's phone for new and escalated assignments

1.3 Non-goals

Explicitly **out of scope** for this project, to keep the solo build tractable:

- **Resident-facing accounts.** Residents keep using the existing Google Form. They never log in.
- **Replacing the IITM EU CMS app.** We record its docket number; we do not integrate with or supersede it. There is no CMS API available to us.
- **Native mobile applications.** The attender interface is a responsive web view.
- **Multi-hostel tenancy.** Single-hostel deployment. The schema does not model a hostel dimension.
- **Payroll or bonus disbursement.** We compute points; money is handled outside the system.
- **A native mobile app store presence.** Push reaches attenders through the installable web app (Web Push), not a Play Store binary.
- **Operating without internet.** The stakeholder confirmed that internet at the hostel office is reliable; the outage we design for is the **cloud platform** being unavailable, not the network. This assumption is load-bearing for Sec. 2.6 and ADR-001.

1.4 Actors

Actor	Count	Device	Authenticated	Responsibilities
Hostel Manager	1	Desktop (office), occasionally mobile	Yes — Google sign-in, allowlisted	Oversight, triage of unclassified complaints, reassignment, penalties, special duties, exports, account provisioning
Attender	~4	Personal Android phone, hostel Wi-Fi / mobile data	Yes — Google sign-in, allowlisted	Receive push alerts, view assigned queue, record CMS docket, mark resolved, claim special duties, view own standing

Actor	Count	Device	Authenticated	Responsibilities
Resident	~hundreds	Any	**No**	Submits the Google Form. Never touches this system directly.
Scheduler	—	—	System	Polls the sheet, ages priorities, runs telemetry scans

1.5 Glossary

Term	Meaning
Complaint	A single maintenance request originating from one Google Form response
Task	The unit of work dispatched to attender(s). One complaint yields one task; special duties are tasks with no complaint
Domain	Category of work: `ELECTRICAL`, `CIVIL`, `RO_PLANT`, `NETWORK`, or `UNCLASSIFIED`
CMS docket	The tracking number issued by the IITM EU Campus Management System when an attender files the issue officially
Triage queue	Holding area for complaints the classifier could not confidently categorize, or that dispatch could not assign
Ledger entry	An immutable row recording a point movement, with actor, cause and effective date
Special duty	A manager-posted ad-hoc task with a custom point value, claimable by any attender
Penalty	A manager-logged infraction carrying a negative point value
SLA	Target resolution time for a complaint, by domain and priority
OCC	Optimistic concurrency control — version-checked updates that fail rather than silently overwrite

1.6 Non-functional requirements

ID	Requirement	Target	Rationale
NFR-1	Scale envelope	≤50 complaints/day, ≤10 concurrent users, ≤50k rows/year	Single hostel, 4 attenders. Sizing everything else follows from this.
NFR-2	Ingestion latency	New Form response visible in the manager dashboard within **5 minutes**	Poll interval, not a real-time requirement
NFR-3	Interactive response time	p95 < 500 ms for queue and dashboard reads	Attenders on mobile data in hostel corridors
NFR-4	Mobile payload	Attender queue initial load < 150 KB	Low-end phones, patchy connectivity
NFR-5	Correctness under concurrency	Zero double-credits, zero lost updates	Core integrity guarantee; see Sec. 7

ID	Requirement	Target	Rationale
NFR-6	Availability	Best-effort. A **cloud app-tier outage** is survivable: the manager runs the same jar on their desktop against the same managed database. Internet is assumed available	Free-tier PaaS sleeps and occasionally fails; outage plan B from proposal Sec. 10
NFR-10	Push delivery	P1 assignment push dispatched within **30 s** of assignment; delivery itself is best-effort	The push service is outside our control, so the in-app queue stays the source of truth
NFR-7	Timezone	All business dates in **Asia/Kolkata**; all timestamps stored UTC	Month boundaries for the bonus cycle must be unambiguous
NFR-8	Auditability	Every state change and point movement is reconstructible	Bonus disputes must be resolvable from the log
NFR-9	Data retention	Complaint text retained 1 year; resident contact detail retained 90 days	Minimize PII exposure (Sec. 11.4)

1.7 Open questions

Pending confirmation from the stakeholder. Each has a working default so no design work is blocked.

Q	Question	Working default
OQ-1	Exact format of the IITM EU CMS docket number	Configurable regex, permissive default `^[A-Za-z0-9/-]{4,32}$`. Validation is a config property, not a code constant.
OQ-2	Per-domain SLA targets	Electrical 8h, RO Plant 12h, Civil 24h, Network 24h (working hours). See Sec. 8.2.
OQ-3	Are attender floor allocations fixed or rotating?	Modelled as time-bounded assignments so both work. See Sec. 3.4.
OQ-4	Exact Google Form column headers and question set	Ingestion maps columns by header name via config, tolerant of reordering. See Sec. 5.2.
OQ-5	Do attenders have domain specialisations, or is everyone a generalist?	Skill weight defaults to neutral; scoring supports specialisation if it exists. See Sec. 6.2.
OQ-6	Working-hours window for SLA aging	08:00–20:00 IST, seven days

1.8 Constraints

- **Solo developer**, roughly nine weeks, alongside coursework. Work is sliced into five delivery phases (Sec. 13), the first of which is a walking skeleton that runs end to end.
- **Every line must be explainable in a viva.** This is a real design constraint, not a formality: where a simpler mechanism gives the same user-visible behaviour, we take the simpler one and record why (ADR-012). Framework features are preferred over hand-rolled equivalents.

- **Free-tier hosting.** Instances sleep when idle; this shapes the scheduler design (Sec. 2.5).
- **No CMS API.** Docket numbers are entered by hand — the system cannot verify them upstream.
- **Low-tech end users.** Attenders may have limited literacy in English and low-end phones. UI must lean on iconography, large tap targets and minimal text entry.

2. System Architecture

2.1 Architectural style

A **modular monolith** deployed as a single Spring Boot executable jar, organised internally into bounded modules with explicit interfaces between them.

This is a deliberate choice over microservices. At NFR-1 scale (≤50 complaints/day, 4 attenders) a distributed system would add network failure modes, deployment complexity and eventual-consistency problems in exchange for scalability we will never need. The modular boundaries below give us the separation-of-concerns benefit without the operational cost, and they are drawn so that a module *could* be extracted later if that ever became necessary.

We use **ports and adapters** at the boundaries where the system talks to the outside world, and nowhere else.

Ingestion is the clearest example. Complaints arrive two ways: polled from the Google Sheet, or uploaded as a CSV when the Sheets API is unavailable (proposal Sec. 10, Plan B). Different sources, but both produce the same thing — a list of raw rows. So we define one interface that says exactly that, `ComplaintSource`, and the two ways of fetching rows become two implementations of it. Everything downstream — deduplication, categorization, dispatch — is written once, against the interface, and never learns which source the rows came from.

The alternative is what usually happens without that boundary: the CSV path grows its own copy of the dedupe and categorization logic. Then a fix lands in one copy and not the other, and the two quietly disagree. The interface is what makes the fallback path *the same path*, which is also why re-uploading the same CSV twice cannot create duplicates — it inherits the guarantee rather than reimplementing it.

2.2 Context diagram

```
graph LR
    R[Resident] -->|submits| GF[Google Form]
    GF -->|appends row| GS[(Google Sheet<br/>responses)]

    subgraph SYS["Hostel Complaint & Duty Tracking System"]
        APP[Spring Boot Application]
    end

    GS -->|Sheets API v4<br/>poll| APP
    CSV[CSV file] -.->|manual upload<br/>fallback| APP
    APP --> DB[(PostgreSQL)]

    M[Hostel Manager<br/>desktop] <-->|HTTPS + SSE| APP
    A[Attender<br/>mobile] <-->|HTTPS| APP
    APP -->|.xlsx| XL[Excel report]
    A -.->|files issue manually,<br/>gets docket no.| CMS[IITM EU CMS App]

    style SYS fill:#eef4ff,stroke:#3b6bb5
    style CMS stroke-dasharray: 5 5
```

The dashed edge to the CMS app is important: there is **no programmatic integration**. The attender files the complaint there by hand and types the resulting docket number into our system. We store it for traceability but can never verify it upstream (see OQ-1).

2.3 Module breakdown

```
graph TD
    subgraph WEB["Web layer"]
        MC[Manager controllers]
        AC[Attender controllers]
        SSE[SSE broadcaster]
```

```
    PUSH[Web Push sender]
end

subgraph CORE["Domain modules"]
    ING[ingestion]
    CAT[categorization]
    DIS[dispatch]
    WF[workflow<br/>state machine]
    LED[ledger]
    ANA[analytics]
    NOT[notification]
    RPT[reporting]
    IAM[identity]
end

subgraph INFRA["Infrastructure"]
    REPO[JPA repositories]
    SHEETS[Sheets adapter]
    CSVA[CSV adapter]
    POI[POI writer]
    SCHED[Scheduler]
end

MC --> WF & DIS & LED & ANA & RPT & IAM
AC --> WF & LED & IAM
ING --> CAT --> DIS --> WF
WF --> LED
WF -->|AFTER_COMMIT| NOT --> SSE
NOT --> PUSH --> WPS[Browser push service]
LED --> NOT
ANA --> REPO
RPT --> POI
ING --> SHEETS & CSVA
SCHED --> ING & ANA & WF
CORE --> REPO --> PG[(PostgreSQL)]
```

Module	Java package	Responsibility	Key invariant it owns
ingestion	`...ingestion`	Fetch raw rows, detect deltas, normalize into complaints	A source row is never turned into two complaints
categorization	`...categorization`	Assign a domain and confidence to complaint text	Low confidence never silently becomes a wrong domain
dispatch	`...dispatch`	Score attenders, produce assignments	Every complaint gets an assignment *or* a recorded triage reason
workflow	`...workflow`	Complaint/task state machine, guards, transitions	Only legal transitions occur; transitions are atomic with their side effects
ledger	`...ledger`	Point movements, leaderboard projection	Append-only; no double-credit; balance is derived, never stored

Module	Java package	Responsibility	Key invariant it owns
analytics	`...analytics`	MTTR, bottlenecks, anomaly detection	Read-only over the ledger and audit tables
notification	`...notification`	Fan-out of domain events to manager SSE streams **and attender Web Push**, on `AFTER_COMMIT`	Delivery is best-effort; never blocks or rolls back a transaction
reporting	`...reporting`	`.xlsx` generation via Apache POI	Reports are point-in-time snapshots, reproducible from stored data
identity	`...identity`	Users, roles, Google allowlist, provisioning	No self-registration; every principal is pre-provisioned
audit	`...audit`	Append-only event log	Written in the same transaction as the change it records

Root package: `in.ac.iitm.hostel.duty`

2.4 Layering rules

```
web (controllers, DTOs, Thymeleaf)
  ↓ may call
application (services, transaction boundaries, orchestration)
  ↓ may call
domain (entities, value objects, state machine, scoring — no Spring, no JPA annotations leaking behaviour)
  ↑ implemented by
infrastructure (JPA repos, Sheets client, POI, SSE)
```

Enforced rules, checked in CI by ArchUnit:

1. `domain` depends on nothing outside itself and the JDK.
2. `web` never touches a repository directly — it goes through `application`.
3. Cross-module calls go through the target module's published service interface, never its internal classes or its repositories.
4. `@Transactional` lives in `application`, never in `web` or in a repository.

Rule 4 matters more than it looks: transaction boundaries are where our concurrency guarantees (Sec. 7) are enforced, so they must be in one predictable place.

2.5 The pipeline

```
sequenceDiagram
    participant S as Scheduler
    participant I as Ingestion
    participant C as Categorizer
    participant D as Dispatch
    participant W as Workflow
    participant N as Notification
    participant M as Manager UI

    S->>I: poll (every 3 min, with lock)
    I->>I: fetch rows → diff against watermark + hashes
    alt new rows found
        I->>C: classify(text)
        C-->>I: domain + confidence
```

```
        alt confidence ≥ threshold
            I->>D: dispatch(complaint)
            D-->>I: assignment or NO_CANDIDATE
        else below threshold
            I->>I: mark UNCLASSIFIED → triage queue
        end
        I->>W: create complaint in NEW / TRIAGE
        W->>N: ComplaintCreated event
        N->>M: SSE push
    else nothing new
        I->>I: no-op, update poll timestamp only
    end
```

Scheduler safety. Free-tier PaaS instances sleep when idle, so `@Scheduled` polling cannot be relied on alone. Three mitigations:

1. A **"Sync Now"** button in the manager dashboard triggers ingestion on demand — this is the guaranteed path, and it also wakes a sleeping instance.
2. On startup the app runs a **catch-up ingestion**, so a wake after sleep loses nothing. Because the delta algorithm (Sec. 5.3) is watermark-and-hash based rather than time based, a long gap is handled identically to a short one.
3. Scheduled jobs take a **database-backed lock** (ShedLock) before running, so if the platform ever runs two instances they cannot double-ingest.

2.6 Deployment topology

```
graph TB
    PHONE[Attender phones] -->|HTTPS| JAR
    DESK[Manager desktop] -->|HTTPS| JAR

    subgraph CLOUD["Cloud — Render / Railway"]
        JAR[app.jar<br/>embedded Tomcat :8080]
    end

    subgraph STANDBY["Standby — manager's desktop"]
        JARL[app.jar<br/>localhost:8080]
    end

    JAR --> PGC[(Managed PostgreSQL<br/>single source of truth)]
    JARL -.->|same database,<br/>over the internet| PGC
    DESK -.->|when the cloud<br/>app tier is down| JARL
    PGC -.->|nightly| BAK[Backup dump]
```

One database, two places the application can run. The standby is the *same jar* with the same `DATABASE_URL`, started on the manager's desktop when Render or Railway is unavailable. Attenders lose access during such an outage — they cannot reach `localhost` — but the manager keeps working, and everything they do lands in the one real database.

Concern	Cloud (primary)	Standby (manager's desktop)
Database	Managed PostgreSQL	**The same** managed PostgreSQL
Reached by	Attenders and manager, over HTTPS	Manager only, on `localhost`
Sheets ingestion	Active, polled	Active — internet is available
Notifications	SSE + Web Push	SSE to the manager; push still delivered
Authentication	Google sign-in, allowlisted	Google sign-in — `http://localhost:8080/...` is a registered redirect URI

Concern	Cloud (primary)	Standby (manager's desktop)
Purpose	Day-to-day operation	Continuity when the app tier is down

This is deliberately the *boring* answer, and it is boring in a useful way. An earlier draft ran the standby on an embedded H2 database seeded from a nightly dump. That required constraining every migration to the SQL subset both engines accept, testing two dialects — and it left an unsolved problem: **work done in the standby had no path back into the cloud database.** Two diverging copies of the ledger is precisely the kind of bug this system exists to eliminate. Pointing both jars at one database does not solve that problem; it means the problem cannot arise (ADR-001).

The residual risk is the managed database itself going down, which no amount of application design fixes. Nightly dumps bound the data loss; for the few hours such an outage lasts the manager falls back to pen and paper, exactly as they do today, and enters the backlog afterwards.

2.7 Cross-cutting concerns

Concern	Approach
Configuration	Spring profiles: `local` (dev) and `cloud` (production and standby — identical config bar the bind address). All tunables (dispatch weights, thresholds, SLA table, poll interval, CMS regex) are `@ConfigurationProperties`, never constants.
Secrets	Environment variables only. The Sheets service-account JSON is supplied as a base64 env var and the OAuth client secret as a plain one — never committed files. `.gitignore` blocks the obvious filenames (Sec. 11.5).
Migrations	Flyway, forward-only, versioned, PostgreSQL-only. Schema changes never hand-edited.
Errors	`@ControllerAdvice` maps domain exceptions to the RFC 7807 problem envelope (Sec. 9.7).
Observability	Spring Actuator health/metrics; structured logging with a correlation ID per request and per ingestion run.
Idempotency	Every state-changing attender action carries a client-generated request key (Sec. 7.4).

3. Data Model

3.1 Design principles

1. **Facts are immutable; state is derived.** Point balances are never stored as a mutable counter. They are the sum of an append-only ledger. This makes double-crediting a constraint violation rather than a race we have to win (Sec. 7).
2. **Every mutable aggregate root carries a `version` column** for optimistic locking.
3. **Audit is written inside the same transaction as the change**, so the log can never disagree with the data.
4. **No hard deletes** on operational data. Rows are tombstoned so the audit trail stays intact.
5. **PostgreSQL only.** Since the standby runs against the same managed database (Sec. 2.6), there is no second engine to accommodate and we may use Postgres features freely.
6. **Prefer a framework table over a hand-rolled one.** Where Spring Security already defines a schema for something, we use it rather than inventing our own (ADR-012).

3.2 Entity–relationship diagram

```
erDiagram
    APP_USER ||--o{ PUSH_SUBSCRIPTION : "receives alerts on"
    APP_USER ||--o{ FLOOR_ASSIGNMENT : "covers"
    APP_USER ||--o{ AVAILABILITY_WINDOW : "declares"
    APP_USER ||--o{ TASK_ASSIGNMENT : "works"
    APP_USER ||--o{ POINT_LEDGER : "earns"

    FLOOR ||--o{ FLOOR_ASSIGNMENT : "assigned in"
    FLOOR ||--o{ COMPLAINT : "located on"

    SOURCE_ROW ||--o| COMPLAINT : "normalizes to"
    INGESTION_RUN ||--o{ SOURCE_ROW : "observed"

    COMPLAINT ||--o| TASK : "yields"
    TASK ||--o{ TASK_ASSIGNMENT : "allocated to"
    TASK ||--o{ POINT_LEDGER : "credits"

    APP_USER ||--o{ AUDIT_EVENT : "acted"
```

3.3 Identity & access

```sql
CREATE TABLE app_user (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    display_name  VARCHAR(128) NOT NULL,
    role          VARCHAR(16)  NOT NULL,   -- MANAGER | ATTENDER
    email         VARCHAR(255) NOT NULL,   -- Google account; the allowlist key
    active        BOOLEAN      NOT NULL DEFAULT TRUE,
    last_login_at TIMESTAMP,
    created_at    TIMESTAMP    NOT NULL,
    created_by    BIGINT       REFERENCES app_user(id),
    version       BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT uq_user_email UNIQUE (email),
    CONSTRAINT ck_user_role  CHECK (role IN ('MANAGER','ATTENDER'))
);
```

There is no credential column anywhere in this schema. Everyone — manager and attenders alike — authenticates with Google, so `email` is purely an *allowlist key*: a sign-in whose verified email has no matching active row is rejected. Nothing secret about any user is stored, so a database breach yields no credential to steal (ADR-002).

There is also **no remember-me table**. Re-authenticating through Google on a device where the account is already signed in is a single tap with no typing — a persistent-token scheme would add a table and a rotation protocol to save an

interaction that already costs nothing.

Two `MANAGER` rows are provisioned from day one. If one Google account becomes inaccessible the other can still administer, including editing the allowlist. This is the break-glass path, and it costs a row rather than a second authentication mechanism (Sec. 11.2).

There is **no self-registration path**. `app_user` rows are created only by a `MANAGER` through the provisioning screen, which is the enforcement point for "only a specific set of accounts may access the system". Google authenticates *who someone is*; this table decides *whether they may in*.

Revoking a lost phone is `active = false`, which blocks the next sign-in immediately and drops the session at its 12-hour expiry. Push subscriptions for that user are revoked in the same action, so the phone stops receiving alerts at once (Sec. 9.6).

If a user loses access to their Google account, a manager edits `email` to whichever account they can use. That is the recovery path, and it is worth being clear that it depends on the person having *some* working Google account — a step that happens outside this system.

3.4 Locations, coverage & availability

```
CREATE TABLE floor (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    wing         VARCHAR(16)  NOT NULL,
    floor_number INTEGER      NOT NULL,
    label        VARCHAR(64)  NOT NULL,
    CONSTRAINT uq_floor UNIQUE (wing, floor_number)
);

-- Current coverage only. Coverage is fixed at Cauvery, not rotating (OQ-3 confirmed).
CREATE TABLE floor_assignment (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    attender_id  BIGINT      NOT NULL REFERENCES app_user(id),
    floor_id     BIGINT      NOT NULL REFERENCES floor(id),
    coverage     VARCHAR(16) NOT NULL,   -- PRIMARY | SECONDARY
    version      BIGINT      NOT NULL DEFAULT 0,
    CONSTRAINT uq_fa      UNIQUE (attender_id, floor_id),
    CONSTRAINT ck_coverage CHECK (coverage IN ('PRIMARY','SECONDARY'))
);
CREATE INDEX ix_fa_lookup ON floor_assignment (floor_id);

CREATE TABLE availability_window (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    attender_id BIGINT       NOT NULL REFERENCES app_user(id),
    status      VARCHAR(16)  NOT NULL,   -- AVAILABLE | ON_LEAVE | OFF_DUTY
    starts_at   TIMESTAMP    NOT NULL,
    ends_at     TIMESTAMP,
    reason      VARCHAR(255),
    recorded_by BIGINT       NOT NULL REFERENCES app_user(id),
    CONSTRAINT ck_avail_status CHECK (status IN ('AVAILABLE','ON_LEAVE','OFF_DUTY'))
);
CREATE INDEX ix_avail_lookup ON availability_window (attender_id, starts_at, ends_at);
```

Coverage is a plain current-state table. An earlier draft made assignments time-bounded (`valid_from` / `valid_to`) so that a past dispatch decision could be replayed against the roster in force at the time — but that put a date filter on every dispatch query to serve a case that does not arise, since coverage at Cauvery is fixed rather than rotating. Explainability is preserved more cheaply: `task_assignment.score_snapshot` (Sec. 3.6) already records the *computed* score that chose an attender, which is the thing a manager actually asks about. Reintroducing the temporal dimension later is an additive migration.

3.5 Ingestion & complaints

```
CREATE TABLE ingestion_run (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
```

```
    source_type   VARCHAR(16) NOT NULL,   -- GOOGLE_SHEETS | CSV_UPLOAD
    trigger_type  VARCHAR(16) NOT NULL,   -- SCHEDULED | MANUAL | STARTUP
    started_at    TIMESTAMP   NOT NULL,
    finished_at   TIMESTAMP,
    rows_seen     INTEGER     NOT NULL DEFAULT 0,
    rows_created  INTEGER     NOT NULL DEFAULT 0,
    rows_skipped  INTEGER     NOT NULL DEFAULT 0,
    status        VARCHAR(16) NOT NULL,   -- RUNNING | SUCCESS | FAILED
    error_detail  VARCHAR(2000),
    triggered_by  BIGINT REFERENCES app_user(id)
);

-- The idempotency anchor for ingestion. One row per observed source record.
CREATE TABLE source_row (
    id             BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    source_type    VARCHAR(16)  NOT NULL,
    source_ref     VARCHAR(128) NOT NULL,  -- sheet id, or upload batch id
    response_key   CHAR(64)     NOT NULL,  -- SHA-256(timestamp || reporter email): STABLE identity
    content_hash   CHAR(64)     NOT NULL,  -- SHA-256 of all normalized cells: change detection
    row_index      INTEGER,                -- positional HINT only, never identity (see 5.3)
    raw_payload    TEXT         NOT NULL,  -- JSON string of the original row
    state          VARCHAR(16)  NOT NULL,  -- ACTIVE (AMENDED reserved, see below)
    revision       INTEGER      NOT NULL DEFAULT 1,
    first_seen_run BIGINT       NOT NULL REFERENCES ingestion_run(id),
    last_seen_run  BIGINT       NOT NULL REFERENCES ingestion_run(id),
    first_seen_at  TIMESTAMP    NOT NULL,
    last_seen_at   TIMESTAMP    NOT NULL,
    complaint_id   BIGINT,
    CONSTRAINT uq_source_row UNIQUE (source_type, source_ref, response_key),
    CONSTRAINT ck_sr_state   CHECK (state IN ('ACTIVE','AMENDED'))
);
CREATE INDEX ix_sr_hash ON source_row (content_hash);

CREATE TABLE complaint (
    id                BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    public_ref        VARCHAR(24)  NOT NULL,   -- e.g. CAU-2026-0417, shown to humans
    source_row_id     BIGINT       REFERENCES source_row(id),
    raw_text          TEXT         NOT NULL,
    reported_at       TIMESTAMP    NOT NULL,
    reporter_room     VARCHAR(32),
    reporter_contact  VARCHAR(128),           -- purged after 90 days (NFR-9)
    floor_id          BIGINT       REFERENCES floor(id),
    location_note     VARCHAR(255),
    docket_no         VARCHAR(64),            -- IITM EU CMS docket; NULL until filed (Sec. 4.3)
    docket_filed_at   TIMESTAMP,
    docket_filed_by   BIGINT       REFERENCES app_user(id),
    domain            VARCHAR(16)  NOT NULL,  -- ELECTRICAL|CIVIL|RO_PLANT|NETWORK|UNCLASSIFIED
    domain_confidence NUMERIC(4,3),           -- 0.000 .. 1.000
    classified_by     VARCHAR(16)  NOT NULL,  -- RULES | MANUAL | FALLBACK
    priority          VARCHAR(8)   NOT NULL,  -- P1 | P2 | P3
    base_priority     VARCHAR(8)   NOT NULL,  -- before aging, kept for explainability
    status            VARCHAR(24)  NOT NULL,
    sla_due_at        TIMESTAMP,
    sla_breached      BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at        TIMESTAMP    NOT NULL,
    updated_at        TIMESTAMP    NOT NULL,
    closed_at         TIMESTAMP,
    version           BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT uq_complaint_ref UNIQUE (public_ref),
    CONSTRAINT ck_domain   CHECK (domain IN
        ('ELECTRICAL','CIVIL','RO_PLANT','NETWORK','UNCLASSIFIED')),
    CONSTRAINT ck_priority CHECK (priority IN ('P1','P2','P3')),
    CONSTRAINT ck_conf     CHECK (domain_confidence IS NULL
                               OR (domain_confidence >= 0 AND domain_confidence <= 1))
);
CREATE INDEX ix_complaint_status ON complaint (status, priority, reported_at);
CREATE INDEX ix_complaint_domain ON complaint (domain, created_at);
CREATE INDEX ix_complaint_sla    ON complaint (sla_due_at, sla_breached);
```

`complaint.raw_text` keeps the resident's original wording verbatim. The classifier's output is stored *alongside* it with a

confidence score and the method used, never in place of it — so a misclassification is always recoverable and the gold-set evaluation in Sec. 12.3 can be run retrospectively over real production data.

3.6 Tasks, dockets and the ledger

```
CREATE TABLE task (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    task_type    VARCHAR(16) NOT NULL,     -- COMPLAINT | SPECIAL_DUTY
    complaint_id BIGINT REFERENCES complaint(id),
    title        VARCHAR(255) NOT NULL,
    description  TEXT,
    point_value  INTEGER     NOT NULL,     -- +2 default for complaints; custom for duties
    shared       BOOLEAN     NOT NULL DEFAULT FALSE,
    status       VARCHAR(24) NOT NULL,
    opens_at     TIMESTAMP,
    closes_at    TIMESTAMP,
    created_by   BIGINT      NOT NULL REFERENCES app_user(id),
    created_at   TIMESTAMP   NOT NULL,
    version      BIGINT      NOT NULL DEFAULT 0,
    CONSTRAINT ck_task_type CHECK (task_type IN ('COMPLAINT','SPECIAL_DUTY')),
    CONSTRAINT ck_task_link CHECK (
        (task_type = 'COMPLAINT'    AND complaint_id IS NOT NULL) OR
        (task_type = 'SPECIAL_DUTY' AND complaint_id IS NULL))
);

CREATE TABLE task_assignment (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    task_id      BIGINT      NOT NULL REFERENCES task(id),
    attender_id  BIGINT      NOT NULL REFERENCES app_user(id),
    assign_method VARCHAR(16) NOT NULL,    -- AUTO | MANUAL | CLAIMED
    assigned_by  BIGINT      REFERENCES app_user(id),
    assigned_at  TIMESTAMP   NOT NULL,
    status       VARCHAR(16) NOT NULL,     -- ACTIVE | RELEASED | COMPLETED
    score_snapshot TEXT,                   -- JSON of the dispatch score that chose them
    version      BIGINT      NOT NULL DEFAULT 0,
    CONSTRAINT uq_task_attender UNIQUE (task_id, attender_id),
    CONSTRAINT ck_asg_method CHECK (assign_method IN ('AUTO','MANUAL','CLAIMED'))
);

CREATE INDEX ix_docket_no ON complaint (docket_no);
```

The docket lives in **three columns on `complaint`**, not a child table. Attenders do mistype numbers on phones, and the correction must be recoverable — but the audit log already records every change with its before and after values (Sec. 3.7), so a separate history table with a `superseded_at` column would store the same fact twice. An earlier draft also gave the docket its own five-state machine including a `VERIFIED` state that, by its own definition, gated nothing. Both are gone (ADR-012); traceability is unchanged, and "who filed which number, when, and what it was corrected from" is answered by one query against `audit_event`.

```
-- Append-only. No UPDATE, no DELETE. Reversals are new rows with negated points.
CREATE TABLE point_ledger (
    id              BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    attender_id     BIGINT       NOT NULL REFERENCES app_user(id),
    task_id         BIGINT       REFERENCES task(id),
    reason_code     VARCHAR(32)  NOT NULL,  -- see table below
    points          INTEGER      NOT NULL,
    effective_date  DATE         NOT NULL,  -- Asia/Kolkata business date (NFR-7)
    period_key      CHAR(7)      NOT NULL,  -- 'YYYY-MM', derived from effective_date
    note            VARCHAR(500),
    idempotency_key VARCHAR(128) NOT NULL,
    reverses_id     BIGINT       REFERENCES point_ledger(id),
    created_by      BIGINT       NOT NULL REFERENCES app_user(id),
    created_at      TIMESTAMP    NOT NULL,
    CONSTRAINT uq_ledger_idem   UNIQUE (idempotency_key),
    CONSTRAINT ck_ledger_reason CHECK (reason_code IN
        ('COMPLAINT_RESOLVED','SPECIAL_DUTY','PENALTY','MANUAL_ADJUSTMENT','REVERSAL'))
);
CREATE INDEX ix_ledger_period  ON point_ledger (period_key, attender_id);
```

```
CREATE INDEX ix_ledger_att     ON point_ledger (attender_id, effective_date);
```

reason_code	Points	Raised by	Idempotency key
COMPLAINT_RESOLVED	+2 (configurable)	Workflow, on transition to RESOLVED	RESOLVE:{task_id}:{attender_id}
SPECIAL_DUTY	task.point_value	Workflow, on special-duty completion	DUTY:{task_id}:{attender_id}
PENALTY	−1 (configurable)	Manager	PENALTY:{uuid} from the request
MANUAL_ADJUSTMENT	any	Manager, with mandatory note	ADJ:{uuid}
REVERSAL	negation of target	Manager	REV:{reverses_id}

This table is the core integrity mechanism. Because the key for an automatic credit is derived deterministically from the task and attender, a retried mobile request — the realistic failure mode on patchy hostel Wi-Fi — collides on `uq_ledger_idem` and is swallowed as a no-op. Double-crediting is not a race we have to win; it is structurally impossible. See Sec. 7.3 for the full argument.

3.7 Audit and classifier data

```
CREATE TABLE audit_event (
    id             BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    occurred_at    TIMESTAMP   NOT NULL,
    actor_id       BIGINT REFERENCES app_user(id),  -- NULL = system
    actor_role     VARCHAR(16),
    entity_type    VARCHAR(32) NOT NULL,
    entity_id      BIGINT      NOT NULL,
    event_type     VARCHAR(48) NOT NULL,
    from_state     VARCHAR(24),
    to_state       VARCHAR(24),
    detail         TEXT,                          -- JSON string
    correlation_id VARCHAR(64)
);
CREATE INDEX ix_audit_entity ON audit_event (entity_type, entity_id, occurred_at);
CREATE INDEX ix_audit_time   ON audit_event (occurred_at);

-- Web Push endpoints. One row per (attender, browser install). See Sec. 9.7.
CREATE TABLE push_subscription (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id      BIGINT       NOT NULL REFERENCES app_user(id),
    endpoint     VARCHAR(512) NOT NULL,   -- push service URL issued by the browser
    p256dh_key   VARCHAR(128) NOT NULL,   -- client public key (payload encryption)
    auth_secret  VARCHAR(64)  NOT NULL,   -- client auth secret
    device_label VARCHAR(64),
    created_at   TIMESTAMP    NOT NULL,
    last_ok_at   TIMESTAMP,
    fail_count   INTEGER      NOT NULL DEFAULT 0,
    revoked_at   TIMESTAMP,
    CONSTRAINT uq_push_endpoint UNIQUE (endpoint)
);
CREATE INDEX ix_push_user ON push_subscription (user_id, revoked_at);

-- Classifier lexicon, editable by the manager without a redeploy (Sec. 5.5)
CREATE TABLE category_term (
    id         BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    domain     VARCHAR(16)  NOT NULL,
    term       VARCHAR(64)  NOT NULL,
    weight     NUMERIC(4,2) NOT NULL DEFAULT 1.00,
    fuzzy      BOOLEAN      NOT NULL DEFAULT TRUE,
    active     BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP    NOT NULL,
```

```
    CONSTRAINT uq_term UNIQUE (domain, term)
);
```

The audit row is written by an `@TransactionalEventListener(BEFORE_COMMIT)` handler, so it commits atomically with the change it describes and the log can never diverge from the data.

Notifications are the mirror image: they fire on **`AFTER_COMMIT`**, so a notification can never describe a transaction that rolled back, and a push service being slow can never hold a database transaction open. An earlier draft routed them through a transactional outbox table with a publisher poller and `Last-Event-ID` replay. For one manager watching one dashboard, refetching current state on reconnect gives the identical user-visible result for a fraction of the machinery (Sec. 9.5, ADR-012).

3.8 Retention

Data	Retention	Mechanism
`complaint.reporter_contact`	90 days	Nightly job nulls the column; audit records the purge
`complaint.raw_text`	1 year	Nightly job replaces with `[purged]`; aggregates survive
`audit_event`, `point_ledger`	Indefinite	Required for bonus dispute resolution
`push_subscription`	Until revoked or 3 consecutive failures	Auto-pruned; endpoint is a device identifier (Sec. 11.4)
`source_row.raw_payload`	90 days	Purged; `content_hash` retained so dedupe still works

Purging `raw_payload` but keeping `content_hash` is what lets retention coexist with idempotency: we can still recognise a previously-seen row without storing what it said.

4. State Machines

4.1 Why this is formalised

Complaint status is the single most-read field in the system: it drives the attender queue, the dashboard, SLA aging, MTTR measurement and the point credit. Leaving it as an informally-updated string is how a ticket ends up `RESOLVED` with no docket, or credited twice.

We therefore define an explicit machine with a **guarded transition table**. The only way to change `complaint.status` is `WorkflowService.transition(complaintId, event, actor, payload)`, which:

1. Loads the aggregate with its `version` (optimistic lock).
2. Looks up `(currentState, event)` in the table; an absent pair throws `IllegalTransition`.
3. Evaluates the guard; failure throws `GuardFailed` with a human-readable reason.
4. Applies the state change, runs the side effects and writes the audit row — all in **one transaction**. Notifications fire separately, on `AFTER_COMMIT` (Sec. 9.5).

There is no other write path. Controllers, the scheduler and the CSV importer all funnel here.

4.2 Complaint lifecycle

```
stateDiagram-v2
    [*] --> NEW : ingested
    NEW --> ASSIGNED : auto_dispatch (candidate found)
    NEW --> TRIAGE : auto_dispatch (no candidate)<br/>or UNCLASSIFIED
    TRIAGE --> ASSIGNED : manager_assign
    TRIAGE --> REJECTED : manager_reject

    ASSIGNED --> ASSIGNED : manager_reassign
    ASSIGNED --> INSPECTING : attender_start
    ASSIGNED --> TRIAGE : attender_release

    INSPECTING --> AWAITING_CMS : defect_confirmed
    INSPECTING --> REJECTED : no_defect_found
    AWAITING_CMS --> IN_PROGRESS : docket_recorded
    IN_PROGRESS --> RESOLVED : mark_resolved
    IN_PROGRESS --> ON_HOLD : block (parts/access)
    ON_HOLD --> IN_PROGRESS : unblock

    RESOLVED --> CLOSED : manager_close / auto_close (7d)
    RESOLVED --> IN_PROGRESS : manager_reopen
    CLOSED --> [*]
    REJECTED --> [*]
```

Transition table

#	From	Event	To	Actor	Guard	Side effects
T1	—	`ingest`	`NEW`	System	Source row not already linked to a complaint	Create complaint, classify, set base priority + SLA due

#	From	Event	To	Actor	Guard	Side effects
T2	`NEW`	`auto_dispatch`	`ASSIGNED`	System	Domain ≠ `UNCLASSIFIED` **and** dispatch returns a candidate	Create task + assignment, snapshot score, emit `TaskAssigned`
T3	`NEW`	`auto_dispatch`	`TRIAGE`	System	Domain `UNCLASSIFIED` **or** no candidate	Record triage reason, emit `TriageRequired`
T4	`TRIAGE`	`manager_assign`	`ASSIGNED`	Manager	Target attender active; domain now set	Set domain (`classified_by = MANUAL`), create task + assignment
T5	`TRIAGE`	`manager_reject`	`REJECTED`	Manager	Reason mandatory	Close complaint, no points
T6	`ASSIGNED`	`manager_reassign`	`ASSIGNED`	Manager	New attender ≠ current; task not yet resolved	Release old assignment, create new one, audit both
T7	`ASSIGNED`	`attender_start`	`INSPECTING`	Assigned attender	Caller holds an `ACTIVE` assignment	Stamp `inspection_started_at`
T8	`ASSIGNED`	`attender_release`	`TRIAGE`	Assigned attender	Reason mandatory	Release assignment, back to triage
T9	`INSPECTING`	`defect_confirmed`	`AWAITING_CMS`	Assigned attender	—	Prompt for docket on mobile UI
T10	`INSPECTING`	`no_defect_found`	`REJECTED`	Assigned attender	Note mandatory	Close; **no points credited**
T11	`AWAITING_CMS`	`docket_recorded`	`IN_PROGRESS`	Assigned attender	Docket matches configured pattern (OQ-1) and is non-blank	Set `docket_no`, `docket_filed_at`, `docket_filed_by`
T12	`IN_PROGRESS`	`block`	`ON_HOLD`	Assigned attender / Manager	Reason mandatory	Pause SLA clock
T13	`ON_HOLD`	`unblock`	`IN_PROGRESS`	Assigned attender / Manager	—	Resume SLA clock, extend due date by hold duration

#	From	Event	To	Actor	Guard	Side effects
T14	IN_PROGRESS	mark_resolved	RESOLVED	Assigned attender	docket_no is set **and** caller assignment is ACTIVE	**Credit points** via ledger (idempotent, Sec. 7.3); mark assignment COMPLETED; emit ComplaintResolved
T15	RESOLVED	manager_close	CLOSED	Manager	—	Stamp closed_at
T16	RESOLVED	auto_close	CLOSED	System	7 days elapsed since resolution	Stamp closed_at
T17	RESOLVED	manager_reopen	IN_PROGRESS	Manager	Reason mandatory	**Reverse the credit** (new REVERSAL ledger row, never a delete)

Notes on specific guards

T11 — the docket gate. mark_resolved (T14) requires a docket to exist. This is the mechanism that makes the proposal's institutional-traceability requirement structurally enforced rather than a matter of attender diligence: you cannot collect +2 points for a complaint you never filed with the Engineering Unit. Because there is no CMS API (Sec. 1.8), we validate *format* only, and the pattern is configuration (OQ-1) so a wrong guess costs a property change, not a redeploy.

T17 — reopen reverses rather than deletes. The ledger is append-only, so a reopen writes a compensating REVERSAL row. The attender's balance falls by 2, and both the original credit and the reversal remain visible in their history. Re-resolving later writes a *new* credit with a new idempotency key (RESOLVE:{task}:{attender}:r{n} where n is the reopen count), so the idempotency guarantee survives the reopen cycle without ever blocking a legitimate re-credit.

T12/T13 — the SLA clock. Hold time is excluded from SLA and from MTTR. Without this, "waiting for the Engineering Unit to send a plumber" would be recorded as attender slowness and would poison the bottleneck analysis in Sec. 8.

4.3 The CMS docket

The docket is **three columns on `complaint`**, not a state machine of its own. Its lifecycle is already implied by the complaint's:

Complaint state	Docket meaning
Before AWAITING_CMS	Not required — the defect has not been confirmed yet
AWAITING_CMS	Confirmed on site, not yet filed with the Engineering Unit
IN_PROGRESS onward	docket_no is set; the complaint has been filed

Corrections overwrite docket_no, and audit_event records the old and new values, who changed it and when. An earlier draft gave the docket its own five-state machine and a history table with a superseded_at column. Both were dropped (ADR-012): the history duplicated the audit log, and the VERIFIED state gated nothing by its own definition — it existed only as a bookkeeping convenience for the manager's periodic reconciliation against the CMS portal, which is now simply a filter on the complaint list.

4.4 Task lifecycle (special duties)

Special duties have no complaint and no docket. They are posted open and claimed.

```
stateDiagram-v2
    [*] --> OPEN : manager posts duty
    OPEN --> CLAIMED : attender claims
    OPEN --> CANCELLED : manager cancels
    OPEN --> EXPIRED : closes_at passes unclaimed
    CLAIMED --> OPEN : attender releases
    CLAIMED --> COMPLETED : attender marks done
    COMPLETED --> CLAIMED : manager reverts (reverses points)
    COMPLETED --> [*]
```

For a **shared** duty (`task.shared = true`) — the common-room overhaul case from the proposal — several attenders may hold `ACTIVE` assignments at once. The duty reaches `COMPLETED` when every active assignee has marked their part done, and **each** assignee is credited once, guarded by their own idempotency key `DUTY:{task_id}:{attender_id}`. The concurrency argument for this is in Sec. 7.5.

4.5 Priority and aging

Priority is not a fixed label; it ages so that old low-priority work eventually surfaces above new noise.

Base priority	Assigned when	Initial SLA (OQ-2)
`P1`	Safety/utility keywords hit (`shock`, `spark`, `no water`, `sewage`, `short circuit`)	Electrical 4h · Other 8h
`P2`	Default for a confidently classified complaint	Electrical 8h · RO 12h · Civil 24h · Network 24h
`P3`	Cosmetic keywords, or explicitly downgraded by the manager	72h

A scheduled job escalates `P3 → P2 → P1` when a complaint has been open past **1.5×** its SLA window without reaching `RESOLVED`, and `base_priority` is retained so the dashboard can always show "P1 (aged from P3)". Escalation is capped at `P1`; hold time (T12) does not count toward it.

5. Ingestion & Categorization

5.1 The hard part

Google Sheets is not an event source. It offers no change feed, no per-row identity and no "give me what changed since X". We poll a mutable document and must derive deltas ourselves, while guaranteeing NFR-1's headline property: **a complaint is never created twice**.

Three properties of the real sheet make the naive approach fail:

1. **Row indices are not stable.** If the manager deletes row 40, every row below shifts up by one. Keying on `(sheet_id, row_index)` would then report the entire tail of the sheet as amended and, depending on the reconciliation rule, could duplicate hundreds of complaints in one poll.
2. **Responses can be edited.** With "edit response" enabled, a resident can change their answer after submission. Content changes; it is still the same complaint.
3. **The instance sleeps.** On free-tier hosting the poller may not run for hours (Sec. 2.5). Any design that depends on "what changed since the last tick" is fragile; the design must be *stateless with respect to elapsed time*.

5.2 Column mapping

Form question wording and column order will change (OQ-4). Ingestion therefore maps by **header name**, configured, not by position:

```yaml
ingestion:
  sheets:
    columns:
      timestamp:  ["Timestamp", "Submitted at"]
      email:      ["Email Address", "Email"]
      room:       ["Room Number", "Room No", "Room"]
      description:["Describe the issue", "Complaint", "Issue"]
      category:   ["Type of issue"]        # optional resident-supplied hint
    required: [timestamp, description]
```

Each logical field lists accepted header aliases. At the start of a run we resolve headers once; an unresolvable **required** column fails the whole run loudly rather than ingesting garbage — a silent partial ingest is far worse than a visible failure, because the manager would have no way to know complaints were being dropped.

5.3 Delta detection

Identity is derived from content, not position:

```
response_key = SHA256( normalize(timestamp) || "\x1f" || normalize(email) )
content_hash = SHA256( join("\x1f", [normalize(cell) for cell in row]) )
```

`response_key` is stable across row shifts and across edits — a Google Form response keeps its submission timestamp and submitter when edited. `content_hash` changes whenever any answer changes. Splitting identity from change-detection is what makes all three failure modes above tractable.

Reconciliation rules

Observation	Meaning	Action
`response_key` unseen	New submission	Insert `source_row`, create complaint (T1)
Known key, same `content_hash`	Already ingested	Skip. Touch `last_seen_*`, nothing else
Known key, different `content_hash`	Response was edited	**Phase 5 feature.** For now: skip and log. See below
Duplicate `response_key` within one batch	Two submissions in the same second by the same person	Disambiguate by appending the in-batch ordinal before hashing

Deletions are not tracked. Once a complaint is ingested it is ours; the sheet is an *inbox*, not a continuing source of truth. An earlier draft detected removed rows (a `VANISHED` state plus a nightly full-sheet reconcile pass) so the manager could be told a row had disappeared. That bought one notification at the cost of a second scan mode, an extra state and a manager review path (ADR-012). Work may already have been done and points credited against a complaint, so we would never have deleted it anyway.

Amendments are deferred to Phase 5. They only occur if the Google Form has "edit response" enabled, which is off by default. Until that is confirmed and the feature is built, an edited row is skipped with a log line — the complaint keeps its original text, which is the safe behaviour. The schema already reserves the `AMENDED` state so adding it later is additive.

Scan strategy

With deletions ignored and amendments deferred, the sheet is append-only from our point of view, so **one fetch mode suffices**:

```
from = max(1, lastRowSeen − OVERLAP)      OVERLAP = 50
fetch rows [from .. end]
```

The overlap absorbs row-index drift if someone does delete a row, and `response_key` makes re-reading rows we already have a no-op regardless. A long gap — the instance slept for six hours, or a manual "Sync Now" after a week — is handled by the same code path: `lastRowSeen` has not moved, so the fetch simply covers more rows. **There is no separate catch-up mode to get wrong.**

Pseudocode

```
function reconcile(rows, run):
    seen = {}
    for (i, row) in rows:
        key = responseKey(row)
        if key in seen: key = responseKey(row, disambiguator = ++seen[key])
        seen[key] = 0

        if sourceRowRepo.exists(sourceRef, key):
            run.rowsSkipped++                          # already ingested
            continue

        sourceRowRepo.save(sourceRow(key, contentHash(row), rowIndex = i, run))
        complaint = normalizeToComplaint(row)          # 5.4
        classify(complaint)                            # 5.5
        workflow.transition(complaint, INGEST)         # T1
        run.rowsCreated++
    run.lastRowSeen = max(rowIndex)
```

Fifteen lines, one branch. That is the entire delta-detection algorithm, and it is worth noting how much of the original complexity came from *features* (amendment tracking, deletion detection) rather than from the hard part — which is identity, and is handled by one `UNIQUE` constraint.

Each row is processed in its **own transaction**. One malformed response therefore cannot abort the whole run; it is recorded on `ingestion_run.error_detail` and the run continues. Given that this is an unattended background job over data we do not control, partial progress is strictly better than all-or-nothing.

Quota and failure budget

Sheets API allows 300 read requests/minute/project. At one request per 3-minute poll we use ~0.1% of quota, leaving ample headroom for manual syncs. Failures back off exponentially (3 → 6 → 12 → 30 min, capped) and after three consecutive failures the dashboard shows a **degraded-ingestion banner** telling the manager to use the CSV fallback. Silence is the failure mode to avoid: the manager must find out from the UI, not by noticing complaints stopped arriving.

5.4 Normalization to a complaint

Field	Derivation
`raw_text`	The description cell, verbatim, never modified
`reported_at`	Parsed form timestamp, interpreted in Asia/Kolkata, stored UTC
`reporter_room`	Regex `\b([A-Z]{0,2}[-\s]?\d{3,4})\b` over the room cell, then the description as fallback
`floor_id`	Derived from room number via a configurable rule (default: first digit of a 3-digit room, first two of a 4-digit). Unmapped → `NULL`, which routes to triage
`reporter_contact`	Email cell; purged after 90 days (NFR-9)
`public_ref`	`CAU-{year}-{zero-padded sequence}`

5.5 Categorization

Approach (ADR-004): a weighted lexicon with fuzzy matching, gated on confidence. Apache OpenNLP document categorization is designed in as an optional second stage (Sec. 5.7) but is *not* on the critical path.

The reasoning is about data, not sophistication. A supervised classifier needs a few hundred labelled examples per class; we do not yet know we will have them, complaint text is short (often under 10 words), frequently misspelled, and often code-mixed Tamil/English transliteration. A weighted lexicon degrades gracefully in exactly those conditions, needs no training data, is editable by the manager without a redeploy, and — decisive for a viva — every decision it makes can be explained by pointing at the terms that fired.

Pipeline

```
raw text
  → lowercase, Unicode NFKC, strip punctuation, collapse whitespace
  → expand abbreviations ("ac"→"air conditioner", "wtr"→"water", "elec"→"electrical")
  → tokenize (whitespace + n-grams up to 3, so phrases can be matched)
  → match against category_term
  → score per domain
  → confidence gate
  → DOMAIN or UNCLASSIFIED
```

Matching and scoring

For each domain d, over matched terms t:

$$S(d) = \sum_{t \in T(d)} w(t) \cdot m(t)$$

$$m(t) = \begin{cases} 1.00 & \text{exact token/phrase match} \\ 0.70 & \text{fuzzy match (Damerau-Levenshtein} \le 2 \text{ on tokens of length} \ge 4, \text{ or Jaro-Winkler} \ge 0.90) \\ 0.00 & \text{otherwise} \end{cases}$$

Multi-word phrases carry higher weights than single tokens because they disambiguate: `water` alone is nearly useless (it appears in RO Plant, Civil and Electrical complaints), while `no water` and `water leaking` are decisive. Phrase matches suppress their constituent single-token matches so a phrase is not double-counted.

Let $S_1 \ge S_2$ be the top two domain scores. Confidence combines **absolute evidence** and **separation**:

$$\text{confidence} = \begin{cases} 0 & \text{if } S_1 < S_{\min} \quad \text{(nothing meaningful matched)} \\ 0 & \text{if } S_1 = 0 \end{cases}$$

```
            (S₁ − S₂) / S₁    otherwise              (margin over the runner-up)

accept if confidence ≥ τ  AND  S₁ ≥ S_min
```

Defaults: `S_min = 1.0`, `τ = 0.40`. Both are configuration, tuned against the gold set (Sec. 12.3).

Two gates rather than one matters. A single absolute threshold accepts a complaint that scores strongly for *two* domains equally — precisely the ambiguous cases where a human should decide. A single margin threshold accepts a complaint where one weak term matched and nothing else did. Requiring both routes exactly the uncertain cases to triage.

Failing to triage, on purpose

Anything not accepted becomes `UNCLASSIFIED` and lands in the manager's triage queue (T3) — the proposal's Risk 2 Plan B. The manager sets the domain in one click, and that correction is recorded. This is a deliberate precision-over-recall stance: a wrongly-auto-assigned complaint sends an attender to the wrong job and erodes trust in the whole system, whereas a triaged one costs the manager three seconds.

Priority extraction

Independent of domain, a separate keyword set raises base priority (Sec. 4.5). `shock`, `spark`, `short circuit`, `sewage`, `no water`, `overflow` → `P1`. These are matched **exactly only** — no fuzzy matching — because a false P1 from a near-miss spelling would distort the whole queue.

5.6 Meeting the 90% accuracy criterion

The acceptance criterion needs a testable definition, so we fix one now:

> Accuracy is measured over the **auto-accepted** subset (confidence $\geq \tau$) of a held-out labelled gold set, as the fraction whose predicted domain equals the human label.

Items routed to triage are excluded from the denominator but tracked separately as **coverage** (the fraction auto-accepted). Reporting both is what makes the number honest — a classifier that accepts nothing would otherwise score 100%. Targets: **accuracy ≥ 90%, coverage ≥ 70%**. Full methodology, dataset construction and the confusion matrix live in Sec. 12.3.

5.7 OpenNLP as a deferred second stage

Once enough manager triage corrections have accumulated, they form a labelled corpus for free — every triage decision is `(text, human label)`. At that point an OpenNLP `DocumentCategorizer` can be trained and consulted **only for items the lexicon routes to triage**, with its own confidence gate. This is upside without risk: the rule engine stays the primary path, so a poorly trained model can only convert triage items into accepted ones, never corrupt a decision the rules were already confident about. Tagged **Extended** in Sec. 13 — it ships only if the schedule allows.

5.8 CSV fallback

Risk 1's Plan B is a drag-and-drop `.csv` upload on the manager dashboard. It is **the same port**:

```java
public interface ComplaintSource {
    String sourceRef();
    SourceType type();
    List<SourceRow> fetch(FetchWindow window);
}
```

`GoogleSheetsSource` and `CsvUploadSource` both implement it, and reconciliation, normalization, classification and dispatch are shared verbatim. The fallback therefore inherits the idempotency guarantee for free: re-uploading the same CSV twice — the obvious thing a flustered user does during an outage — produces zero duplicate complaints, because `response_key` collides.

6. Dispatch Heuristic

6.1 Problem statement

Given a classified complaint c and the set of attenders A, choose the attender who should do the work — accounting for who covers that floor, who is actually on duty, and who is already overloaded. The proposal requires this be a genuine heuristic rather than round-robin, and that it be **formally specified** so its behaviour is testable.

Requirements the formulation must satisfy:

R	Requirement
R1	Always terminates with either an assignment or an explicit, recorded triage reason
R2	Deterministic — identical inputs give identical output, including tie-breaks
R3	Explainable — the manager can see *why* an attender was chosen
R4	No starvation — one attender cannot absorb everything while others idle
R5	Tunable without a redeploy

6.2 Formulation

Stage 1 — hard eligibility filter

Availability is a **gate, not a score**. An attender on leave must never be chosen, no matter how well they score elsewhere; folding it into a weighted sum would let a strong floor match outvote the fact that someone is not at work.

```
E = { a ∈ A :  a.active
            ∧ availabilityAt(a, now) = AVAILABLE
            ∧ ¬ alreadyAssignedTo(a, c) }
```

If `E = ∅` → **no assignment**, complaint → `TRIAGE` with reason `NO_AVAILABLE_ATTENDER` (T3). This is the proposal's "all assigned floor attenders unavailable" edge case, and it is handled by construction rather than by a special branch.

Stage 2 — score the eligible set

```
Score(a, c) = w_f · Floor(a, c)  +  w_l · Load(a)
```

Two terms, each normalised to **[0, 1]**, weights summing to 1, so `Score ∈ [0, 1]` and each weight reads directly as "how much does this factor matter".

Floor proximity — `Floor(a, c)`

Situation	Value
`PRIMARY` coverage of the complaint's floor	**1.00**
`SECONDARY` coverage of that floor	**0.70**
Covers an adjacent floor in the same wing ($\lvert\Delta\rvert = 1$)	**0.40**
Same wing, $\lvert\Delta\rvert \geq 2$	`max(0.10, 0.35 − 0.05·|Δ|)`
Different wing	**0.10**

Non-zero floors for distant attenders are intentional: when the primary is unavailable, someone still has to go, and a zero would make the whole term uninformative in exactly that case.

If `c.floor_id IS NULL` (room number unparseable, OQ-4) the term is undefined. Rather than guess, the complaint goes to `TRIAGE` with reason `UNMAPPED_LOCATION` — dispatching on a location we could not read is worse than asking.

Workload — `Load(a)` Higher score = *more* free capacity, so the term pushes work toward idle attenders.

```
L(a) = Σ  π(t)          over tasks t currently ACTIVE for a
       t                (status ∈ {ASSIGNED, INSPECTING, AWAITING_CMS, IN_PROGRESS})

where π(P1) = 3,  π(P2) = 2,  π(P3) = 1        (ON_HOLD tasks count 0)

Load(a) = 1 − min(1, L(a) / L_cap)              L_cap default 8
```

Weighting by priority rather than counting tickets reflects reality: three P1 electrical faults is not the same workload as three P3 cosmetic complaints. `ON_HOLD` work is excluded because the attender is genuinely not blocked by it (Sec. 4.2, T12).

Default weights

Weight	Value	Rationale
`w_f` floor	**0.60**	The stakeholder's stated primary criterion
`w_l` load	**0.40**	Strong enough to override floor when someone is badly overloaded

Both, plus `L_cap` and the proximity table, are `@ConfigurationProperties` (R5). Sec. 12 pins their behaviour with tests that assert *relationships* ("an overloaded primary loses to an idle secondary") rather than exact float outputs, so tuning does not break the suite.

Two terms that were cut An earlier draft added a **domain skill** term and a **continuity** term. Both were removed (ADR-012), and the reasoning is worth keeping because it generalises:

- **Skill** defaulted to a neutral `0.5` for every attender, because we do not know whether attenders specialise (OQ-5). A constant added identically to every candidate **cannot change the ranking** — it was arithmetic that looked like intelligence. If specialisations are ever confirmed, reintroducing the term is a one-line addition to the formula.
- **Continuity** (has this attender worked this room recently?) carried 10% weight and required a 14-day lookback query per candidate. The signal it captures is real, but it is better used where it is actually actionable: Sec. 8.3's chronic-defect detector already flags repeat faults at the same location, which tells the manager to *replace the fixture* rather than merely routing the next complaint to the same person.

Stage 3 — select

```
best = argmax Score(a, c)
        a∈E
```

Tie-break chain, applied in order (R2, R4):

1. Fewest tasks assigned **today** — the anti-starvation term
2. Longest time since last assignment
3. Lowest `app_user.id` — guarantees full determinism

Then: create `task`, create `task_assignment` with `assign_method = AUTO`, and persist the full score breakdown into `task_assignment.score_snapshot` as JSON (R3):

```
{"chosen": 7, "score": 0.850,
 "components": {"floor": 1.0, "load": 0.625},
 "weights": {"floor": 0.60, "load": 0.40},
 "runnersUp": [{"attenderId": 9, "score": 0.460}],
 "tieBreak": null}
```

The snapshot is what makes "why did Ravi get this?" answerable months later, and it is what the dashboard renders in the assignment tooltip.

Capacity overflow

If every eligible attender is at or over `L_cap`, we still assign to the best-scoring one — work cannot simply vanish — but emit a `CapacityExceeded` notification so the manager sees the queue is saturated. Overflowing visibly beats triaging silently.

6.3 Shared tasks

For `task.shared = true` (common-room overhaul, furniture transport), dispatch selects the **top *k*** by score instead of the top 1, where *k* is set by the manager when posting the duty. Each selected attender gets their own `task_assignment` row and, on completion, their own ledger credit under their own idempotency key. The concurrency consequences are Sec. 7.5.

6.4 Degenerate cases

Case	Detection	Outcome
No attender available	`E = ∅`	`TRIAGE` / `NO_AVAILABLE_ATTENDER`
Room number unparseable	`floor_id IS NULL`	`TRIAGE` / `UNMAPPED_LOCATION`
Floor exists but nobody covers it	All `Floor = 0.10`	Assign anyway (least-loaded wins), warn `UNCOVERED_FLOOR`
Complaint `UNCLASSIFIED`	Domain gate	Never reaches dispatch — triaged at T3
Every attender at capacity	`∀a: L(a) ≥ L_cap`	Assign best, notify `CapacityExceeded`
Exact score tie	Equal `Score`	Tie-break chain, Sec. 6.2 stage 3
Manager disagrees	—	`manager_reassign` (T6); both assignments audited

Every row terminates (R1). There is no path where a complaint silently has no owner — the invariant the dispatch module is responsible for (Sec. 2.3).

6.5 Why greedy, and not optimal assignment

A global optimum over a batch (Hungarian algorithm, min-cost flow) is the textbook answer and is **deliberately rejected** here:

- Complaints arrive **one at a time** from the poll. A batch is typically 1–3 items, and the optimal assignment of a 1-item batch is the greedy one.
- With $|A| = 4$, the theoretical gain over a whole day is negligible, while the cost is real: batch optimisation would mean holding complaints back to accumulate a batch, adding latency to exactly the P1 cases that matter most.
- Greedy is explainable (R3). "Ravi scored 0.78, Kumar 0.50" is something the manager can argue with. "The global cost matrix minimised at this permutation" is not.

Complexity is $O(|A| \cdot \log|A|)$ per complaint — trivially within NFR-3. This trade-off is recorded as ADR-005.

With two terms and four candidates the whole heuristic is a dozen lines of pure Java with no framework involvement — which is exactly what makes it easy to unit-test and to defend.

7. Points Ledger & Concurrency Control

7.1 What can actually go wrong

The proposal calls for "strict transactional isolation and optimistic locking to prevent race conditions and double-crediting". Before choosing mechanisms, here is the concrete threat model — five races that will genuinely occur in this deployment:

#	Race	Realistic trigger	Consequence if unhandled
C1	**Double resolve**	Attender taps *Resolve*, the response is lost on hostel Wi-Fi, they tap again	+4 points for one job — corrupts the bonus
C2	**Lost update**	Manager reassigns a task at the same moment the attender resolves it	Credit goes to the wrong person, or the reassignment silently vanishes
C3	**Shared-task double completion**	Two attenders on a shared duty mark done simultaneously	Task completion logic runs twice; over-notification, possible double close
C4	**Over-claim**	Special duty with `k = 1` claimed by two attenders in the same second	Two people show up; one is credited for work they did not get
C5	**Torn leaderboard read**	Dashboard reads while credits are being written	Standings briefly inconsistent with the ledger

These are not hypothetical. C1 in particular is *the expected case* on a phone in a hostel corridor, not an exotic edge.

7.2 Mechanisms

We use four layered mechanisms rather than one, because each race has a different shape:

M	Mechanism	Addresses
M1	Optimistic locking (`@Version`) on every mutable aggregate	C2, C3
M2	`UNIQUE (idempotency_key)` on `point_ledger`	C1
M3	Client-supplied `Idempotency-Key` on state-changing requests	C1 (at HTTP level, before it reaches the domain)
M4	Short pessimistic lock on the task row for claims	C4
M5	Leaderboard derived by query, never stored	C5

A note on the layering: M2 alone is sufficient to make double-crediting impossible. M3 exists so the *user experience* of a retry is a clean success rather than a swallowed conflict, and M1 exists for the non-ledger state that also needs protecting.

7.3 Why double-crediting is structurally impossible

This is the central correctness argument of the system.

The credit for resolving a complaint is written with a key derived **deterministically from facts that identify the work**, not from anything about the request:

```
String key = "RESOLVE:" + task.getId() + ":" + attender.getId()
           + (reopenCount > 0 ? ":r" + reopenCount : "");
```

Crediting is then:

```
@Transactional
void credit(Task task, Attender a, int points, String reasonCode, String key) {
    try {
        ledgerRepo.saveAndFlush(new LedgerEntry(a, task, reasonCode, points, key, ...));
    } catch (DataIntegrityViolationException dup) {
        // uq_ledger_idem — this exact credit already exists. Not an error.
        log.info("Credit {} already applied; no-op", key);
    }
}
```

The guarantee does not rest on transaction ordering, isolation level, retry logic, or on the application checking-then-inserting (which is itself a race). It rests on a **database uniqueness constraint**, which the database enforces atomically no matter how many concurrent transactions attempt the insert. Whether the duplicate arrives 20 milliseconds later on a retried HTTP request or 20 minutes later from a queued action replayed after a signal drop, the outcome is identical: one credit.

The `:r{n}` suffix handles the reopen cycle (T17). After a reopen the reopen counter increments, so a legitimate re-resolution produces a *different* key and is correctly credited again — the constraint blocks duplicates without ever blocking genuine repeat work.

Consequence: `attender.points` does not exist as a column anywhere in the schema. There is no counter to increment, therefore no lost-update on a counter, therefore nothing to reconcile. A balance is a `SUM` over immutable facts.

7.4 Idempotency at the HTTP boundary

Every state-changing attender action carries a key generated on the client:

```
POST /api/attender/tasks/42/resolve
Idempotency-Key: 6f1c2b90-...        # UUID v4, persisted in localStorage until ACKed
```

The mobile UI generates the key **before** the first send and reuses it for every retry of that same user intent. Server-side, a filter records `(key, endpoint, status, response)` for 24 hours; a replay returns the stored response without re-executing.

This is what makes the flaky-connection story work end to end. The attender taps *Resolve*, loses signal, the service worker queues the request, and when connectivity returns it replays with the same key. Whether the original request actually reached the server or not, the attender sees one success and the ledger holds one credit.

7.5 Shared tasks

For `task.shared = true`, each assignee has an independent `task_assignment` row and an independent idempotency key `DUTY:{task_id}:{attender_id}`. Two attenders completing simultaneously therefore credit *different* keys — no contention, both correct.

The contended part is the **task's own completion**, which depends on all assignees being done:

```
@Transactional
void markMyPartDone(long taskId, long attenderId, String idemKey) {
    Task task = taskRepo.findById(taskId).orElseThrow();          // loads @Version
    TaskAssignment mine = assignmentRepo.findActive(taskId, attenderId)
                          .orElseThrow(NotAssigned::new);
    mine.setStatus(COMPLETED);
    ledger.credit(task, attenderId, task.getPointValue(), "SPECIAL_DUTY", idemKey);

    if (assignmentRepo.countActiveIncomplete(taskId) == 0) {
        workflow.transition(task, TASK_COMPLETED);                // bumps task.version
    }
}
```

Under concurrency both transactions read `task.version = v`. The one that commits second finds the version has moved and throws `ObjectOptimisticLockingFailureException`. It is retried once transparently; on retry it re-reads, sees its peer already completed the task, and skips the transition. Its own credit — already keyed uniquely — is unaffected either way. C3 resolved.

7.6 Claiming a duty (C4)

Claiming is a genuine read-modify-write on a count (`active claims < k`), and OCC handles it, but the retry-on-conflict path is user-visible and confusing for someone standing in a corridor. Since contention is trivially low — at most four people, claiming a handful of duties a week — we take a **short pessimistic lock** and give the loser an immediate, clear answer:

```
@Transactional
void claim(long taskId, long attenderId) {
    Task t = taskRepo.findByIdForUpdate(taskId);      // SELECT ... FOR UPDATE
    if (t.getStatus() != OPEN) throw new AlreadyClaimed();
    if (assignmentRepo.countActive(taskId) >= t.getSlots()) throw new NoSlotsLeft();
    assignmentRepo.save(new TaskAssignment(t, attenderId, CLAIMED));
}
```

The lock is held for microseconds. `uq_task_attender` (Sec. 3.6) additionally makes a double-claim by the *same* attender impossible regardless. Choosing pessimistic locking here, against the general preference for OCC elsewhere, is recorded as ADR-006 — the deciding factor is UX clarity at negligible contention, not performance.

7.7 Optimistic locking and conflict UX

`@Version` guards `complaint`, `task`, `task_assignment`, `floor_assignment` and `app_user`.

Situation	Handling
Background/system job hits a conflict	Retry up to 3× with jitter, then log and skip
Attender action hits a conflict	Retry once transparently; then HTTP `409` with a plain-language message
Manager action hits a conflict	HTTP `409`, UI shows the current state and asks for confirmation

C2 concretely: the manager reassigns while the attender resolves. Both mutate `task_assignment`. One commits; the other gets a 409. If the attender lost, they are told "this task was reassigned to someone else" — which is true, and far better than silently crediting a person who no longer owns the work.

Isolation level: `READ COMMITTED` (the PostgreSQL default). We deliberately do not raise it. Every invariant we depend on is enforced by a uniqueness constraint or a version check, both of which hold at `READ COMMITTED`. `SERIALIZABLE` would add serialization failures and retry complexity to buy a guarantee we have already obtained more cheaply. Recorded as ADR-007.

7.8 The leaderboard

A derived projection, never stored state:

```
SELECT u.id, u.display_name,
       COALESCE(SUM(l.points), 0)                                    AS total,
       COALESCE(SUM(CASE WHEN l.points > 0 THEN l.points END), 0)    AS earned,
       COALESCE(SUM(CASE WHEN l.points < 0 THEN l.points END), 0)    AS deducted,
       COUNT(CASE WHEN l.reason_code = 'COMPLAINT_RESOLVED' THEN 1 END)  AS resolved
  FROM app_user u
  LEFT JOIN point_ledger l ON l.attender_id = u.id AND l.period_key = :period
 WHERE u.role = 'ATTENDER' AND u.active
 GROUP BY u.id, u.display_name
 ORDER BY total DESC, resolved DESC, u.display_name;
```

`LEFT JOIN` so an attender with zero activity still appears — a missing row would read as a bug to the manager. Ordering is fully deterministic through the tie-breaks.

At NFR-1 scale (~50k ledger rows/year, ~600/month per period) this is an index scan over a few hundred rows and needs **no caching, no materialized view and no denormalised counter**. Adding one would trade away the very property that makes the ledger correct. If volume ever justified it, the right move would be a materialized view refreshed on commit — not a mutable counter.

7.9 Month boundaries

`period_key` is derived from `effective_date`, which is the **Asia/Kolkata business date** (NFR-7), not the UTC instant. A task resolved at 02:00 IST on 1 October belongs to October, which is what the manager expects; storing UTC and grouping naively would have silently filed it under September.

A penalty for late September behaviour, logged on 2 October, is entered with an explicit `effective_date` of the September date. It lands in the September period, the audit records both the effective date and the entry date, and if the September leaderboard has already been exported the manager is warned that a closed period is being amended. We allow it — the alternative is a manager who cannot correct a genuine mistake — but never silently.

8. Analytics & Telemetry

8.1 Where the numbers come from

Analytics reads only from `audit_event` and `point_ledger`. Because every status change is written through the single transition path (Sec. 4.1) with `from_state`, `to_state` and `occurred_at`, the lifecycle of any complaint is fully reconstructible from the log — we never needed to add special-purpose measurement columns, and no metric can drift out of sync with the operational data.

Stage durations are the intervals between consecutive transitions:

Stage	From → To	What it actually measures
Dispatch latency	`NEW` → `ASSIGNED`	Our system's own responsiveness
Pickup latency	`ASSIGNED` → `INSPECTING`	How long before an attender starts
Inspection latency	`INSPECTING` → `AWAITING_CMS`	Time to confirm the defect on site
Filing latency	`AWAITING_CMS` → `IN_PROGRESS`	Delay in getting a CMS docket
Repair latency	`IN_PROGRESS` → `RESOLVED`	The actual fix, excluding holds
Closure latency	`RESOLVED` → `CLOSED`	Manager confirmation lag

Time spent in `ON_HOLD` is subtracted from the enclosing stage (Sec. 4.2, T12). Without that, "waiting three days for the Engineering Unit to send a plumber" would be recorded as attender slowness and would point the bottleneck analysis at exactly the wrong place.

8.2 MTTR and bottleneck detection

Resolution time = `RESOLVED.occurred_at − complaint.reported_at − holdDuration`.

The proposal asks for MTTR, which conventionally means the arithmetic mean. We compute and display the mean, but present the **median as the headline figure**, with p90 alongside:

At ~20 complaints/day, one complaint that sat over a long weekend moves the mean by hours while the typical experience is unchanged. Reporting only the mean would make the dashboard react to noise and would quietly mislead the manager about what is normal. Showing mean, median and p90 together costs nothing and makes the skew visible rather than hidden.

Bottleneck identification, per domain d:

$$\text{For each stage } s\text{: } \quad \text{share}(s, d) = \frac{\text{median duration of } s \text{ in } d}{\Sigma \text{ median duration of all stages in } d}$$

$$\text{bottleneck}(d) = \operatorname{argmax} \text{share}(s, d), \quad \text{reported only when share} \geq 0.35$$

The 0.35 floor exists so we do not report a "bottleneck" when time is spread evenly — a stage that is 26% of a five-stage pipeline is not a bottleneck, it is arithmetic. Each result is displayed with its sample size, and suppressed entirely below `n = 10` for that domain.

SLA targets driving breach metrics (OQ-2, pending confirmation):

Domain	P1	P2	P3
Electrical	4h	8h	72h
RO Plant	6h	12h	72h
Civil	8h	24h	72h
Network	8h	24h	72h

Measured in working hours (08:00–20:00 IST, OQ-6). These are configuration, not constants.

8.3 Anomaly detection

The honest constraint

This deserves stating plainly, in the design doc rather than after the fact: with **4 attenders and ~20 complaints/day**, we are doing statistics on small samples. A textbook z-score over a 30-day window has $n \approx 600$ overall but only $n \approx 150$ per domain and $n \approx 5$ per domain-day. Ordinary mean-and-standard-deviation outlier detection on data this thin and this skewed will produce frequent false alarms, and a dashboard that cries wolf is one the manager stops reading — which is a worse outcome than having no anomaly detection at all.

Three design responses follow from this:

1. **Robust statistics only.** Median and MAD, never mean and σ, for outlier detection. Both have a 50% breakdown point; the mean has 0%.
2. **Minimum sample gates.** Every detector declares an n below which it emits nothing.
3. **Evidence with every alert.** Each alert carries its statistic, its threshold and its n, so a sceptical manager can judge it rather than trust it.

Detectors

A1 — Prolonged resolution latency. Modified z-score (Iglewicz & Hoaglin):

```
MAD  = median( |xᵢ − median(x)| )
Mᵢ   = 0.6745 · (xᵢ − median(x)) / MAD
flag if Mᵢ > 3.5
```

Reference set: same domain, trailing 90 days, resolved complaints. **Gate: n ≥ 12.** If `MAD = 0` (possible when most complaints resolve in an identical rounded time) fall back to flagging `xᵢ > 3 × median`, since the modified z-score is undefined there.

A2 — Chronic location / band-aid repair. No statistics at all, just counting:

```
flag room r if  ≥3 complaints in the same domain from r within 30 days
severity HIGH if ≥3 within 14 days
```

Exact counting is *more* reliable than a statistical test at this sample size, and this detector directly serves the proposal's "chronic fixture defects / band-aid repairs" requirement. It is also the most operationally actionable output the system produces: three electrical faults in room 214 in a fortnight needs replacing, not another patch.

A3 — Arrival volume surge. Poisson tail test against an EWMA baseline:

```
L_t = α·x_t−₁ + (1−α)·L_t−₁          α = 0.3
flag if  P(X ≥ x_t | Poisson(L_t)) < 0.01
```

Poisson rather than Gaussian because daily arrival counts are small non-negative integers — exactly what Poisson models and where a normal approximation is worst. **Gate: ≥14 days of history.** Separate baselines per wing, so a burst localised to one wing (a failed riser, a water outage) is caught even when the hostel total looks normal.

A4 — SLA breach cluster. Flag when breaches in a rolling 7 days exceed 2× the trailing 8-week rate for that domain. **Gate: ≥4 weeks of history.**

A5 — Workload imbalance. Flag when `max(L(a)) / mean(L(a)) > 2.0` across active attenders for 3 consecutive days — this is a check on our own dispatch heuristic (Sec. 6) as much as on the attenders.

Alert storage

```sql
CREATE TABLE analytics_alert (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    detector     VARCHAR(8)   NOT NULL,      -- A1..A5
    severity     VARCHAR(8)   NOT NULL,      -- INFO | WARN | HIGH
    subject_type VARCHAR(24)  NOT NULL,      -- COMPLAINT | ROOM | DOMAIN | ATTENDER | WING
    subject_ref  VARCHAR(64)  NOT NULL,
    summary      VARCHAR(255) NOT NULL,
```

```
    evidence      TEXT         NOT NULL,      -- JSON: statistic, threshold, n, window
    detected_at   TIMESTAMP    NOT NULL,
    acknowledged_at TIMESTAMP,
    acknowledged_by BIGINT REFERENCES app_user(id),
    dedupe_key    VARCHAR(128) NOT NULL,
    CONSTRAINT uq_alert_dedupe UNIQUE (dedupe_key)
);
```

`dedupe_key` (e.g. `A2:ROOM:214:ELECTRICAL:2026-09`) means a nightly scan re-raising the same finding updates rather than duplicates it — otherwise a genuine month-long chronic-defect condition would generate thirty identical alerts and bury everything else.

Detectors run on a nightly `@Scheduled` scan, holding the same ShedLock as ingestion (Sec. 2.5). They are pure reads over `audit_event`; they never write to operational tables.

8.4 Dashboard visualizations

Chart.js, all fed from JSON endpoints (Sec. 9.4):

Visualization	Type	Answers
Arrival velocity	Line, 30 days, per domain	"Are complaints trending up?"
Resolution funnel	Stacked bar by stage	"Where does time go?"
MTTR by domain	Bar + median/p90 error markers	"Which category is slowest?"
Attender workload	Horizontal bar, live	"Who is swamped right now?"
SLA breach distribution	Donut by domain	"What are we failing?"
Leaderboard	Ranked table, live via SSE	"Who is ahead this month?"
Anomaly feed	Card list, severity-sorted	"What needs my attention?"

Every chart that renders a derived statistic shows its **sample size**. A chart drawn from six data points must not look as authoritative as one drawn from six hundred, and the *n* is the cheapest possible way to convey that.

8.5 Explicit limitations

Recorded here so they can be stated honestly in the viva rather than discovered by a grader:

- Anomaly detection is **advisory**, never automated action. Nothing is reassigned or escalated on the strength of a detector firing.
- A1 assumes resolution times within a domain are drawn from one distribution. A domain with two genuinely distinct populations (5-minute fuse swaps and 3-day rewiring jobs) will flag the long tail as anomalous. Mitigation: segment by priority once volume allows.
- A3's baseline is contaminated by the surges it is meant to detect — a sustained high period becomes the new normal within roughly $1/\alpha \approx 3$ days.
- The first four weeks after go-live produce almost no alerts by design, because every gate is unmet. This is correct behaviour and should not be read as a defect.

9. API & Data Contracts

9.1 Shape of the web layer

A hybrid, chosen to match the two very different clients:

- **Thymeleaf server-rendered pages** deliver the page shell, navigation and first paint. For the attender on a low-end phone this matters — the queue is usable before any JavaScript runs, which is the difference between a working tool and a spinner in a corridor with two bars of signal.
- **JSON endpoints under `/api/**`** serve dynamic data: chart series, live queue refreshes, and every state-changing action.
- **SSE at `/api/manager/stream`** pushes live updates to the manager dashboard.
- **Web Push** delivers assignment alerts to attender phones even when the app is closed (Sec. 9.6).

All JSON endpoints are versionless (`/api/...`) — this is a single-deployment internal tool with no third-party consumers, so a version prefix would be ceremony without a payer.

9.2 Conventions

Aspect	Rule
Timestamps	ISO-8601 UTC with `Z` (`2026-09-08T14:32:00Z`). Clients render in Asia/Kolkata
Dates	`YYYY-MM-DD`, always the Asia/Kolkata business date
Money/points	Integers. No floats anywhere in the ledger
Nulls	Omitted from responses rather than sent as `null`
Casing	`camelCase`
Paging	`?page=0&size=25`, envelope `{content, page, size, totalElements, totalPages}`
Auth	Session cookie (`HttpOnly; Secure; SameSite=Lax`), established via Google OIDC for every user
CSRF	Spring Security tokens on all non-GET form posts
Idempotency	`Idempotency-Key` header **required** on attender state-changing POSTs (Sec. 7.4)

9.3 Attender API (mobile)

Method	Path	Purpose	Transition
`GET`	`/api/attender/me`	Profile, month points, rank	—
`GET`	`/api/attender/queue`	Assigned active tasks, priority-ordered	—
`POST`	`/api/attender/tasks/{id}/start`	Begin inspection	T7
`POST`	`/api/attender/tasks/{id}/confirm-defect`	Defect is real	T9
`POST`	`/api/attender/tasks/{id}/no-defect`	Nothing wrong found	T10
`POST`	`/api/attender/tasks/{id}/docket`	Record CMS docket number	T11
`POST`	`/api/attender/tasks/{id}/resolve`	Mark fixed → **+2 points**	T14
`POST`	`/api/attender/tasks/{id}/hold`	Block with reason	T12
`POST`	`/api/attender/tasks/{id}/release`	Hand back to triage	T8
`GET`	`/api/attender/duties`	Open special duties	—
`POST`	`/api/attender/duties/{id}/claim`	Claim a slot	Sec. 7.6
`POST`	`/api/attender/duties/{id}/complete`	Mark my part done	Sec. 7.5

Method	Path	Purpose	Transition
`GET`	`/api/attender/standing`	Point history, this period	—
`GET`	`/api/attender/push/key`	VAPID public key for subscription	—
`POST`	`/api/attender/push/subscribe`	Register this device for push	—
`POST`	`/api/attender/push/unsubscribe`	Stop push on this device	—
`POST`	`/api/attender/push/test`	Send a test notification to self	—

`GET /api/attender/queue`

```
{
  "attenderId": 7,
  "generatedAt": "2026-09-08T09:14:02Z",
  "tasks": [
    {
      "taskId": 412,
      "complaintRef": "CAU-2026-0417",
      "priority": "P1",
      "agedFrom": "P2",
      "domain": "ELECTRICAL",
      "title": "Fan not working, sparking sound",
      "room": "214",
      "floor": "Wing A, Floor 2",
      "status": "ASSIGNED",
      "reportedAt": "2026-09-08T08:41:00Z",
      "slaDueAt": "2026-09-08T12:41:00Z",
      "slaState": "AT_RISK",
      "docket": null,
      "nextActions": ["start", "release"],
      "points": 2
    }
  ],
  "counts": {"assigned": 3, "inProgress": 1, "onHold": 0}
}
```

`nextActions` is computed **server-side from the state machine**, not inferred by the client. The mobile UI renders exactly the buttons this array names. This keeps Sec. 4's transition table as the single source of truth for what is legal — the client cannot offer an action the server would reject, so an attender never taps a button that fails.

`POST /api/attender/tasks/{id}/resolve`

```
POST /api/attender/tasks/412/resolve
Idempotency-Key: 6f1c2b90-8e44-4c1a-9f2b-3d5e7a1c0b22
Content-Type: application/json

{"note": "Replaced fan capacitor and switch"}
```

```
{
  "taskId": 412,
  "status": "RESOLVED",
  "resolvedAt": "2026-09-08T11:02:41Z",
  "pointsAwarded": 2,
  "periodTotal": 34,
  "alreadyApplied": false
}
```

`alreadyApplied: true` on a replay — the request succeeded, no second credit was written (Sec. 7.3). The client shows the same success screen either way; the field exists for diagnostics, not for the user.

9.4 Manager API

Method	Path	Purpose
GET	/api/manager/summary	Dashboard header counts
GET	/api/manager/complaints	Filter/paginate all complaints
GET	/api/manager/triage	Unclassified + unassignable queue
POST	/api/manager/complaints/{id}/classify	Set domain manually (T4)
POST	/api/manager/complaints/{id}/assign	Assign from triage (T4)
POST	/api/manager/complaints/{id}/reassign	Move to another attender (T6)
POST	/api/manager/complaints/{id}/reject	Reject with reason (T5)
POST	/api/manager/complaints/{id}/reopen	Reopen, reverses credit (T17)
POST	/api/manager/complaints/{id}/act-as	**Assisted admin log** (Risk 3 Plan B)
POST	/api/manager/penalties	Log a −1 infraction
POST	/api/manager/duties	Post a special duty
POST	/api/manager/adjustments	Manual point adjustment, note required
GET	/api/manager/leaderboard?period=2026-09	Standings
GET	/api/manager/analytics/mttr	MTTR + median + p90 by domain
GET	/api/manager/analytics/arrivals	Arrival velocity series
GET	/api/manager/analytics/funnel	Stage-share breakdown
GET	/api/manager/analytics/workload	Live per-attender load
GET	/api/manager/alerts	Anomaly feed
POST	/api/manager/alerts/{id}/ack	Acknowledge an alert
POST	/api/manager/ingestion/sync	**Sync Now** (Sec. 2.5)
POST	/api/manager/ingestion/csv	CSV fallback upload (Risk 1 Plan B)
GET	/api/manager/reports/daily.xlsx	Daily Excel export
GET	/api/manager/reports/weekly.xlsx	Weekly Excel export
GET/POST	/api/manager/users	List / provision accounts
PATCH	/api/manager/users/{id}/email	Point the allowlist at a different Google account
POST	/api/manager/users/{id}/deactivate	Lost-phone / departure response
POST	/api/manager/availability	Mark an attender on leave

`act-as` is worth noting: it lets the manager perform an attender's transition during roll-call when that attender cannot use their phone. It executes the identical transition through the identical state machine, but writes `actor_id = manager, on_behalf_of = attender` to the audit log. The points still go to the attender — the manager is the typist, not the earner — and the audit record makes that distinction permanent.

`POST /api/manager/penalties`

```
{"attenderId": 9, "points": -1, "reasonCode": "LATE_ARRIVAL",
 "effectiveDate": "2026-09-08", "note": "Reported 45 min late, no prior intimation"}
```

`note` is mandatory for every negative adjustment. A point deduction affects someone's bonus; an unexplained one is indefensible in a dispute, which is precisely when the record gets read.

9.5 SSE contract

```
GET /api/manager/stream
Accept: text/event-stream

event: ComplaintCreated
data: {"complaintId":903,"ref":"CAU-2026-0418","domain":"CIVIL","priority":"P2","room":"310"}
```

```
event: TaskResolved
data: {"taskId":412,"attenderId":7,"attenderName":"Ravi","points":2,"periodTotal":34}
```

Event	Fired when
`ComplaintCreated`	Ingestion creates a complaint (T1)
`TriageRequired`	Complaint needs manager attention (T3)
`TaskAssigned`	Dispatch assigns (T2)
`TaskStarted` / `TaskResolved`	Attender progress (T7 / T14)
`DocketRecorded`	CMS number entered (T11)
`DutyClaimed` / `DutyCompleted`	Special duty progress
`PenaltyLogged`	Manager logs an infraction
`AlertRaised`	Anomaly detector fires
`IngestionDegraded`	Poll failing repeatedly (Sec. 5.3)
`PushDeliveryFailed`	An attender's device stopped accepting push (Sec. 9.6)

Events are published from an `@TransactionalEventListener(AFTER_COMMIT)` handler, so a notification can never describe a transaction that rolled back, and a slow subscriber can never hold a database transaction open. A heartbeat comment every 20 s keeps free-tier proxies from reaping an idle connection.

On reconnect the client refetches current state (`/api/manager/summary` plus the visible lists) rather than replaying missed events. An earlier draft wrote every event to a transactional outbox table and replayed by `Last-Event-ID` after a dropped connection. For one manager watching one dashboard, a refetch produces the identical screen — and it removes a table, a publisher poller, event-id bookkeeping and a replay cap from the system (ADR-012). SSE here is a *latency optimisation over polling*, not a delivery guarantee, and the design is much easier to reason about once that is stated plainly.

9.6 Push notification contract

Attenders receive alerts on their phone even when the app is closed, using the **Web Push protocol with VAPID** (ADR-011). The same service worker that backs the offline queue (Sec. 10.2) receives them, so no second client-side runtime is introduced.

Subscription flow

```
1. Attender logs in on their phone, taps "Enable alerts"
2. Browser asks for notification permission        (one-time, OS-level)
3. Client GETs /api/attender/push/key              → VAPID public key
4. Client calls pushManager.subscribe(...)         → endpoint + p256dh + auth
5. Client POSTs those to /api/attender/push/subscribe
6. Server stores a push_subscription row keyed on the endpoint
```

```
POST /api/attender/push/subscribe
Content-Type: application/json

{
  "endpoint": "https://fcm.googleapis.com/fcm/send/dQw4w9...",
  "keys": {"p256dh": "BNcRdreALRFXTkOOUHK1EtK2wtaz5Ry4YfYCA_0QTpQtUbVlUls0VJXg7A8u-Ts1XbjhazAkj7I99e8QcYP7DkM=",
           "auth":   "tBHItJI5svbpez7KI4CCXg=="},
  "deviceLabel": "Ravi's phone"
}
```

`endpoint` is unique (Sec. 3.7), so re-subscribing the same browser updates rather than duplicates the row — a browser reissues its endpoint periodically, and without that constraint an attender would accumulate stale subscriptions and receive the same alert several times.

Notification types

Only events an attender must **act on** are pushed. Everything else stays in the app.

Trigger	Title	Body	Priority
`TaskAssigned` (P1)	! Urgent: Room 214	Electrical — sparking sound. Due in 4h	high
`TaskAssigned` (P2/P3)	New task: Room 118	Civil — door latch broken	normal
Priority escalated to P1	T Now urgent: Room 305	RO Plant — overdue	high
`SpecialDutyPosted`	New duty available (+5)	Hostel night preparation	normal
`TaskReassigned` away	Task reassigned	Room 118 moved to Kumar	normal
SLA breach imminent (30 min)	T Due soon: Room 214	Not yet resolved	high

Deliberately not pushed: point credits, leaderboard changes, manager dashboard activity, anomaly alerts. Notifying someone that they earned the points they just earned is noise, and an attender who learns to swipe our notifications away will miss the P1 that matters.

Payload

Encrypted per the Web Push spec (`aes128gcm`); the push service never sees the plaintext.

```
{
  "title": "! Urgent: Room 214",
  "body": "Electrical — sparking sound. Due in 4h",
  "tag": "task-412",
  "data": {"taskId": 412, "url": "/m/tasks/412"},
  "actions": [{"action": "open", "title": "Open"}],
  "renotify": false
}
```

`tag` collapses repeat notifications for the same task, so an escalation replaces the original alert rather than stacking on it.

Delivery semantics

Push is **best-effort and advisory** (NFR-10). The in-app queue remains the source of truth — an attender who never enables notifications loses nothing but convenience, and no state transition depends on a push being delivered. Push sends therefore happen from an `@TransactionalEventListener(AFTER_COMMIT)` handler — never inside the transaction that caused them, so a slow push service can never hold a database transaction open or roll one back.

Response from push service	Meaning	Action
`201` / `200`	Accepted for delivery	Record `SENT`
`404` / `410`	Subscription expired	Revoke the row permanently
`429`	Rate limited	Back off, retry per `Retry-After`
`413`	Payload too large	Truncate body, retry once
`5xx`	Push service down	Retry ×3 with backoff, then give up

Outcomes are logged and counted on `push_subscription.fail_count`; there is no per-attempt table. Three consecutive failures revoke the subscription and surface a "notifications not reaching this device" note on the manager's attender-admin screen — a phone that silently stopped receiving alerts is exactly the failure the manager needs told about, not one to discover during a P1.

9.7 Error model

RFC 7807 problem details:

```
{
  "type": "https://hostel.iitm/errors/illegal-transition",
  "title": "Action not allowed in current state",
  "status": 409,
  "detail": "Task 412 is RESOLVED; 'start' is only valid from ASSIGNED.",
  "instance": "/api/attender/tasks/412/start",
  "code": "ILLEGAL_TRANSITION",
  "currentState": "RESOLVED"
}
```

code	HTTP	Meaning
ILLEGAL_TRANSITION	409	Event not legal from the current state (Sec. 4.2)
GUARD_FAILED	422	Legal transition, unmet precondition (e.g. resolve with no docket)
OPTIMISTIC_LOCK	409	Concurrent modification; client should refresh
ALREADY_CLAIMED	409	Duty taken (Sec. 7.6)
NO_SLOTS_LEFT	409	Shared duty full
NOT_ASSIGNED	403	Caller does not hold an active assignment
INVALID_DOCKET	422	Docket fails the configured pattern (OQ-1)
INGESTION_COLUMN_MISSING	422	Required sheet column unresolvable (Sec. 5.2)
NOT_PROVISIONED	403	Google sign-in succeeded, but the email is not on the allowlist (Sec. 11.2)
VALIDATION_FAILED	400	Bean-validation failure; `errors[]` lists fields

`detail` is written to be shown directly to the user. On the attender's phone there is no developer to interpret a stack trace, and "This task was already resolved" is the entire message that person needs.

10. Frontend Design

10.1 Two clients, two very different users

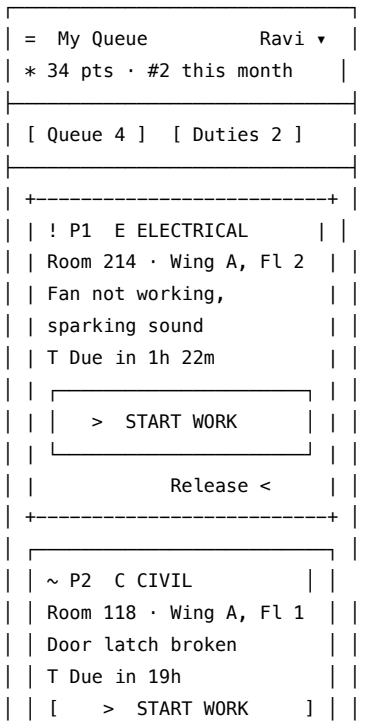
	Manager (desktop)	Attender (mobile)
Device	Office desktop, 1440px+	Low-end Android, 360px
Session	Long, seated, focused	30 seconds, standing, one hand
Network	Office LAN	Patchy hostel Wi-Fi / mobile data
Literacy	Comfortable with spreadsheets	May have limited English literacy
Needs	Density, overview, control	One task, one obvious action

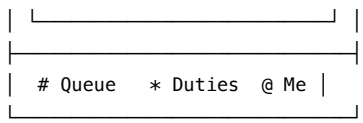
These are not the same application with a different breakpoint. They are two layouts sharing a backend, and the attender view is designed **mobile-only** — it is never used on a desktop, so it carries no desktop compromise.

10.2 Attender mobile

Design rules, all driven by the table above:

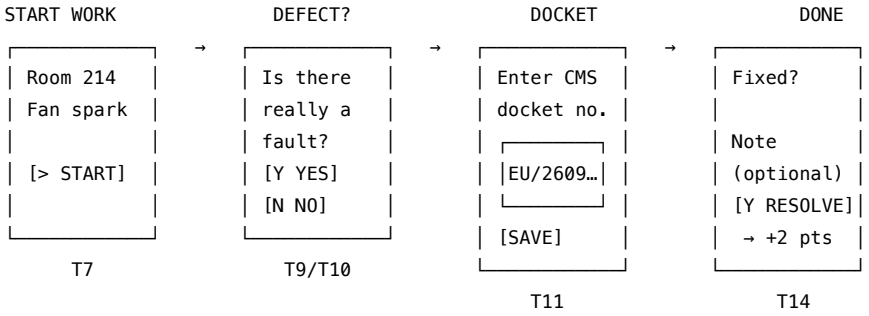
1. **Never more than one primary action visible per task.** The state machine already knows what comes next (`nextActions`, Sec. 9.3); the UI renders that one button large.
2. **Tap targets ≥ 48×48 px**, single-column, no horizontal scrolling.
3. **Icon + colour carry meaning, text confirms it.** Domain and priority are recognisable without reading.
4. **Minimal typing.** The only free-text entries in the whole flow are the CMS docket number and an optional resolution note.
5. **Optimistic UI.** Actions apply locally and show immediately. A failed send is retried with the same idempotency key (Sec. 7.4), so a retry on poor signal is always safe. Full offline queuing (IndexedDB replay) is backlog, Sec. 13.8 — the idempotency guarantee it relies on ships in Phase 1.
6. **Push alerts for new work.** The attender should not have to poll the app to find out a P1 was assigned to them (Sec. 9.6). Permission is requested *after* the first successful login, never on a cold first visit — a permission prompt before any value is shown is the reliable way to get it denied forever.





The resolution flow

Four screens, each one decision:



The docket screen cannot be skipped — it is the guard on T14 (Sec. 4.2), and putting it inline in the flow rather than as a separate chore is what makes institutional traceability happen without nagging.

Offline behaviour

A service worker caches the shell and the current queue, so the app opens and shows the last known queue even on a bad connection. Actions that fail to send are retried with the same idempotency key and show a **"~ retrying"** badge.

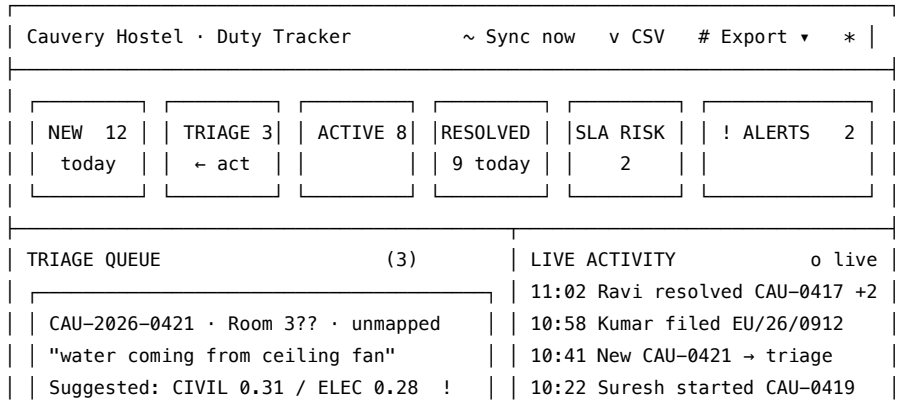
Full offline queuing — storing actions in IndexedDB and replaying them on reconnect — is backlog (Sec. 13.8). It is a small addition precisely because the hard part is already done: replay is safe at any distance in time because the ledger key is derived from the work, not the request (Sec. 7.3).

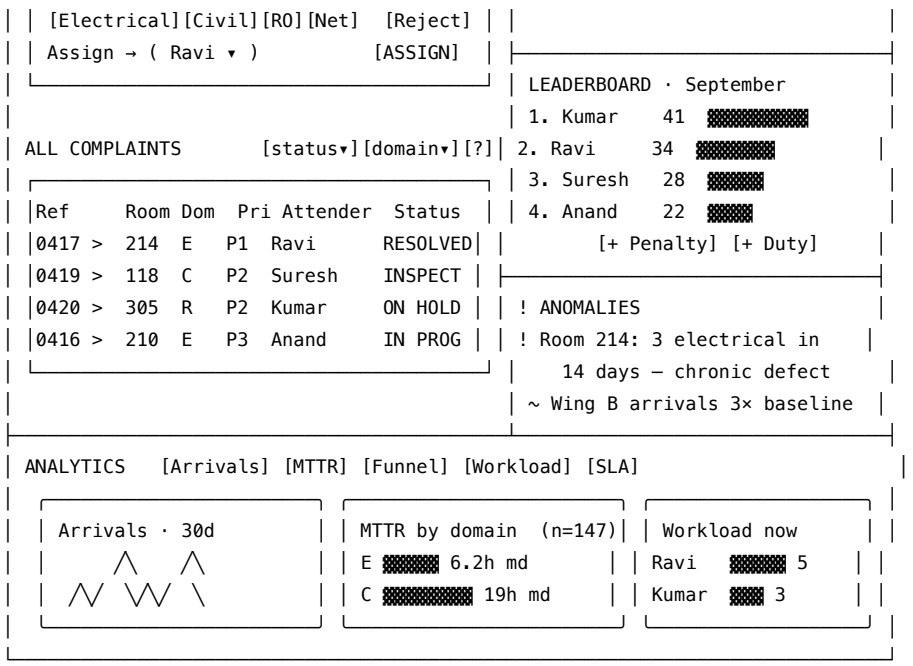
The **same service worker** receives Web Push (Sec. 9.6), which is why the app is a PWA at all: one registration serves both offline replay and notifications. Tapping a notification deep-links straight to `/m/tasks/{id}`, so the path from buzz to action is a single tap.

Notification settings

`/m/me` carries a simple toggle set — all alerts, urgent (P1) only, or off — plus a "send test notification" button. Attenders who find alerts intrusive can dial them down rather than revoking browser permission entirely, which would be irreversible from inside the app.

10.3 Manager desktop





Design notes

Triage is placed above the complaint list, not in a tab. It is the only part of the dashboard that *requires* the manager's action, and the entire value proposition is reducing their daily 30–45 minutes. Anything requiring action goes where the eye lands first.

The triage card shows the classifier's runner-up scores (`CIVIL 0.31 / ELEC 0.28`). This turns an opaque failure into useful information: the manager sees *why* the system was unsure, which makes the one-click correction feel like collaboration rather than cleanup after a dumb machine.

Live activity and the leaderboard are SSE-driven. No polling, no refresh button — the proposal's "real-time duty updates" requirement.

Every derived statistic shows its `n` (Sec. 8.4).

10.4 Screens

Screen	Route	Audience
Login	`/login`	Both — a single "Sign in with Google" button; role comes from the allowlist
Manager dashboard	`/`	Manager
Complaint detail	`/complaints/{ref}`	Manager — full audit timeline, score breakdown, docket history
Attender queue	`/m`	Attender
Task detail / flow	`/m/tasks/{id}`	Attender
Special duties	`/m/duties`	Attender
My standing	`/m/me`	Attender — point history, month rank, notification settings
Attender admin	`/admin/attenders`	Manager — provisioning, allowlist email, leave, floor coverage
Reports	`/reports`	Manager — date range, export

Screen	Route	Audience
Settings	`/settings`	Manager — lexicon, weights, thresholds, SLA table

The **complaint detail** screen is the explainability surface: the full audit timeline, the dispatch score snapshot (Sec. 6.2) rendered as a readable breakdown, the docket history including superseded entries, and every ledger movement attached to the task. If the manager ever asks "why did this happen", one screen answers it.

10.5 Styling & accessibility

Tailwind CSS (utility classes, no separate design system to maintain solo). Chart.js for visualizations.

- WCAG AA contrast throughout; **colour is never the only signal** — priority carries an icon and a text label as well as a hue, which also covers colour-blind users
- Semantic HTML, labelled form controls, visible focus rings
- Mobile: no fixed-position overlays that a soft keyboard can trap
- Charts have accessible data-table fallbacks

10.6 Performance

Target	Approach
Attender first load < 150 KB (NFR-4)	Server-rendered shell, Tailwind purged, no framework runtime, Chart.js loaded **only** on manager routes
p95 < 500 ms (NFR-3)	Indexed queries (Sec. 3), no N+1 (fetch joins on queue and dashboard), pagination everywhere
Chart payloads	Pre-aggregated server-side; the browser never receives raw rows

11. Security & Privacy

11.1 Threat model

This is an internal tool for one hostel with five accounts, deployed on public cloud hosting. Naming what we are and are not defending against keeps the controls proportionate:

Threat	In scope	Response
Opportunistic internet scanning of a public URL	[yes]	Auth on everything, no anonymous endpoints
A student discovering the URL and trying to log in	[yes]	Allowlist — a valid Google sign-in still fails without a provisioned row
An attender viewing or altering another attender's points	[yes]	Role + ownership checks on every action
An attender gaming the point system	[yes]	Docket gate on credit, append-only ledger, audit
A lost or shared attender phone	[yes]	`active = false` blocks the next sign-in; push subscriptions revoked at once; session expires within 12 h
Resident PII exposure	[yes]	Minimisation, retention limits (Sec. 11.4)
Targeted attack by a skilled adversary	[no]	Out of scope; no assets justify it
DDoS	[no]	Free-tier platform's problem

The most *likely* real incident here is mundane: a shared or lost phone. The controls above are weighted accordingly — and note that with no credential stored anywhere in this system, a lost phone exposes a *session*, not a secret.

11.2 Authentication

Per ADR-002: **one mechanism for everyone — Google sign-in, admitted by an allowlist.** No passwords, no PINs, no credential of any kind stored by this system.

	Every user
Mechanism	Google OIDC (`openid, email, profile`)
Credential stored	**None**
Admission decided by	Verified email present and `active` in `app_user`
Role	`app_user.role`, never anything Google asserts
Session	12 h idle timeout
Re-authentication	One tap — account already signed in on the device

The allowlist is the access-control boundary

Google establishes *who someone is*; it has no opinion on whether they may use this system. That decision is ours and is made in one place — a custom `OidcUserService` that resolves the verified email against `app_user` and throws if there is no matching active row. A sign-in from any other Google account, institute or personal, is rejected without creating anything.

Emails are normalised (trimmed, lower-cased) before comparison and matched only against the `email_verified` claim, never a self-asserted address.

Why one mechanism rather than two

An earlier draft kept a local password for the manager, on the grounds that they need the standby during a cloud outage, that a demo must work regardless of network, and that they are the support desk for everyone else. All three were

re-examined once "outage" was confirmed to mean the cloud platform rather than the network:

- **Standby.** Google permits `http://localhost` redirect URIs — the standard pattern for local development and installed apps. Registering `http://localhost:8080/login/oauth2/code/google` alongside the cloud URI means the standby signs in normally.
- **Demo.** The application is hosted on Render. If the network is down the demo is down regardless of how authentication works, so a local password buys nothing in the scenario it was meant for.
- **Support desk.** This one is real, and it is answered by a **row, not a mechanism** (below).

Collapsing to one mechanism deletes the `password_hash` and `username` columns, the BCrypt encoder, the login form and controller, password rate limiting, and one of two `SecurityFilterChain` branches. What remains is a single sentence: *every user authenticates with Google and is admitted by the allowlist.*

Break-glass

Two `MANAGER` rows are provisioned from day one — the manager's own account and a second one (the hostel office's shared account, or a personal Gmail). If either becomes inaccessible, the other can still administer, including editing the allowlist.

Beyond that, the developer holds database access, so `UPDATE app_user SET email = ?` is the ultimate recovery path. That is written into the runbook rather than dressed up as a feature. It is worth noting this is no worse than a password would have been: a forgotten manager password ends in exactly the same place, with one more mechanism to have built.

What this costs, stated plainly

Exposure	Assessment
Dependency on Google availability	Nobody can sign in if Google OIDC is unreachable. The application is cloud-hosted, so a network failure already takes the system down independently of authentication
Not integration-tested end to end	The authorization-code flow is Spring Security's code. What we wrote — the allowlist decision — is unit-tested directly; other tests use `@WithMockUser` (Sec. 12.1)
OAuth client secret; two redirect URIs	One environment variable (Sec. 11.5) and a one-time console setup. Scopes are non-sensitive, so no Google verification review is required
Recovery is partly external	A user who loses their Google account needs *some* working account before a manager can repoint the allowlist
Email addresses stored	Personal data, recorded in Sec. 11.4

Revoking a lost device is `active = false`, which blocks the next sign-in immediately and revokes that user's push subscriptions in the same action; the session expires within 12 hours.

11.3 Authorization

Role-based, enforced at the service layer (not only in controllers) via `@PreAuthorize`. Roles come from `app_user.role` at sign-in, never from anything Google asserts — Google tells us *who*, we decide *what they may do*:

Capability	Manager	Attender
View all complaints	[yes]	[no] (own assignments only)
Trigger ingestion / upload CSV	[yes]	[no]
Classify, assign, reassign, reject, reopen	[yes]	[no]
Start / inspect / docket / resolve	[yes] (via `act-as`)	[yes] (own assignments only)

Capability	Manager	Attender
Claim special duties	[no]	[yes]
Log penalties, adjustments, special duties	[yes]	[no]
View leaderboard	[yes] (full)	[yes] (ranks + own detail)
View another attender's point detail	[yes]	[no]
Provision users, edit the allowlist, deactivate accounts	[yes]	[no]
Export reports	[yes]	[no]
Edit lexicon, weights, SLA table	[yes]	[no]

Ownership is checked in addition to role: an attender acting on a task must hold an `ACTIVE task_assignment` for it (`NOT_ASSIGNED` → 403). Role alone is never sufficient for a task action — otherwise any attender could resolve any other attender's work and collect the points.

11.4 PII and data minimisation

Data	Why we hold it	Retention (NFR-9)
Resident email	Contacting the reporter if details are unclear	90 days, then nulled
Room number	**Operationally essential** — dispatch depends on it	1 year
Complaint text	Diagnosis, classification, chronic-defect detection	1 year, then `[purged]`
Attender name	Assignment, leaderboard	Life of employment
Google email (all users)	Allowlist key, and identifies the actor in the audit log	Life of employment
Push endpoint + keys	Delivering assignment alerts	Until revoked or 3 failures; a device identifier, so treated as personal data

We **do not** collect resident names or phone numbers even when the Form offers them — the mapped column set (Sec. 5.2) simply omits them. The cheapest way to protect data is not to hold it.

Aggregates survive purging: MTTR, arrival counts and chronic-defect detection all continue to work on rows whose text has been redacted, because they depend on timestamps, domain and room — not on what the resident wrote.

11.5 Secrets

Secret	Handling
Google service-account JSON	Base64 in `GOOGLE_SA_KEY` env var. **Never a file in the repo.**
Database credentials	`DATABASE_URL` env var, injected by the platform
Session signing key	`APP_SECRET` env var, generated per environment
Google OAuth client	`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` env vars. Authenticates the application to Google; **not** a user credential
VAPID key pair	`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` env vars. The private key authenticates us to push services; rotating it invalidates every subscription, so it is generated once per environment

`.gitignore` blocks `**/google-service-account*.json`, `**/credentials.json`, `.env*` and `application-local.yml` (Sec. repo root). CI runs **Gitleaks** on every PR so a committed secret fails the build rather than reaching `main`. The Sheets service account is granted **read-only** access to the single response sheet — if it leaks, the blast radius is reading complaints we already have.

11.6 Application security

Control	Implementation
SQL injection	JPA/parameterised queries only; no string-concatenated SQL
XSS	Thymeleaf escapes by default; `th:utext` is banned and checked in review
CSRF	Spring Security tokens on all state-changing form posts
Clickjacking	`X-Frame-Options: DENY`
Transport	HTTPS enforced (`Strict-Transport-Security`); platform-terminated TLS
Headers	CSP, `X-Content-Type-Options: nosniff`, `Referrer-Policy`
Upload safety	CSV only, ≤5 MB, ≤5000 rows, content-type and header validated before parsing
Push payload privacy	Payloads are end-to-end encrypted (`aes128gcm`) per the Web Push spec, so Google's or Mozilla's push service relays ciphertext it cannot read. Bodies still avoid resident identifiers — a lock-screen preview is visible to anyone holding the phone, so notifications name a room, never a person
Dependencies	OWASP Dependency-Check in CI; Dependabot enabled
Error leakage	RFC 7807 responses only; stack traces never returned to a client

11.7 Audit as a security control

`audit_event` (Sec. 3.7) is append-only and covers sign-ins (including rejected ones, with the email that was refused), provisioning, every state transition, every point movement, `act-as` actions, and configuration changes.

Two properties make it trustworthy rather than decorative: it is written **in the same transaction** as the change it records, so it cannot silently diverge; and there is **no application code path that updates or deletes an audit row** — the JPA entity exposes no setters and the repository no delete method. Since the ledger is also append-only, an attender's monthly bonus can be reconstructed from first principles and defended line by line in a dispute, which is the situation this control actually exists for.

12. Test Plan

12.1 Strategy

Tests are written **with the phase that introduces the behaviour** (Sec. 13), not batched at the end. Phase 1 ships with the ledger and auth tests; the concurrency suite lands with Phase 4's shared tasks. A phase is not complete until its tests are green.

Level	Tool	Scope	Target
Unit	JUnit 5, AssertJ	Pure domain logic — scoring, classification, state machine, statistics	~70% of tests; no Spring context
Slice	`@DataJpaTest`, `@WebMvcTest`	Repositories against real Postgres; controller contracts	~15%
Integration	`@SpringBootTest` + Testcontainers	Full pipeline, transactions, concurrency	~10%
Concurrency	JUnit + `CountDownLatch` / `ExecutorService`	The races in Sec. 7.1	Explicit, named suite
Architecture	ArchUnit	Layering rules (Sec. 2.4)	Continuous
Manual/UAT	Stakeholder	Acceptance criteria (Sec. 12.6)	Per milestone

Authentication is tested where the logic actually is. The OAuth2 authorization-code flow is Spring Security's code and is not re-tested; what *we* wrote is the allowlist decision, which is a plain unit test over `OidcUserService` — known email admits with the right role, unknown email rejects, inactive user rejects, case and whitespace variants normalise. Every other test authenticates with `@WithMockUser`. Since there is now only one authentication mechanism, this is the *entire* authentication test surface.

All database tests run against real PostgreSQL via Testcontainers, never an in-memory substitute. The properties we most need to verify — unique-constraint behaviour under concurrent inserts, optimistic-lock failures, `SELECT ... FOR UPDATE` — are precisely the ones an H2 stand-in would fake convincingly and wrongly. Coverage floor: **75% line, 90% on `domain`**, enforced by JaCoCo in CI.

12.2 Fixtures

Fixture	Source	Size	Used by
F1 — Real anonymised complaints	Cauvery Hostel logs, PII replaced with dummy data	~200 rows	Classifier gold set, realistic dispatch
F2 — Synthetic edge cases	Hand-authored	~60 rows	Corner-case suite (Sec. 12.4)
F3 — Concurrency workload	Generated	4 attenders × 25 tasks	Race tests (Sec. 12.5)
F4 — Volume/telemetry	Generated, 90 days	~1800 complaints	Analytics, anomaly detectors
F5 — Malformed sheet	Hand-authored	~30 rows	Ingestion robustness

Anonymisation of F1 happens once, at extraction, before the data ever enters the repository. A committed fixture is effectively permanent, so the scrub cannot be a later step: emails → `resident{n}@example.com`, names removed, room numbers remapped through a fixed permutation that preserves floor structure (so dispatch tests stay meaningful) while breaking identifiability. The mapping table is not committed.

12.3 Categorization gold set

The proposal's "≥90% accuracy" needs a defensible methodology, not a spot check.

Construction. From F1, 200 complaints labelled by hand into `ELECTRICAL`, `CIVIL`, `RO_PLANT`, `NETWORK`, `AMBIGUOUS`. Split **70/30 into development and holdout**. Lexicon terms and thresholds are tuned *only* against development; the holdout is scored once per milestone. Tuning against the holdout would produce a number that means nothing.

Metrics. Reported together, never in isolation:

```
coverage  = auto-accepted / total
accuracy  = correct predictions / auto-accepted          ← the ≥90% criterion
recall_d  = correctly predicted as d / truly d
```

Plus a per-domain confusion matrix.

Coverage is reported alongside accuracy because **accuracy alone is trivially gameable**: a classifier that refuses to classify anything scores 100%. Targets: **accuracy ≥ 90%, coverage ≥ 70%** on the holdout. Complaints labelled `AMBIGUOUS` by the human labeller are expected to land in triage; if one is auto-accepted, that is counted as an error.

Continuous evaluation. Every manager triage correction in production is `(text, true label)` — a free growing labelled set. A monthly re-scoring job reports drift, and this corpus is also what would eventually make the OpenNLP stage (Sec. 5.7) viable.

12.4 Corner-case suite

Every case named in the proposal's verification plan, plus what the design surfaced:

#	Case	Expected
E1	Ambiguous text scoring two domains equally	`UNCLASSIFIED` → triage, both scores shown
E2	Identical simultaneous submissions (different residents)	Two complaints — distinct `response_key` (Sec. 5.3)
E3	Byte-identical resubmission by the same resident	One complaint — `response_key` collides
E4	Same CSV uploaded twice	Zero duplicates
E5	Unallocated / unparseable floor number	`TRIAGE` / `UNMAPPED_LOCATION`
E6	All attenders marked unavailable	`TRIAGE` / `NO_AVAILABLE_ATTENDER`, nothing lost
E7	Floor exists, nobody covers it	Assigned to least-loaded, `UNCOVERED_FLOOR` warning
E8	Every attender over `L_cap`	Assigned to best, `CapacityExceeded` notification
E9	Sheet row deleted mid-month	Ignored; the complaint is unaffected (Sec. 5.3)
E10	Sheet row edited after ingestion	Skipped and logged; original text retained. *(Amendment handling is backlog, Sec. 13.8)*
E11	Row deleted causing all indices to shift	Zero spurious amendments (Sec. 5.1)
E12	Required column renamed	Run fails loudly with `INGESTION_COLUMN_MISSING`
E13	Empty / whitespace-only complaint text	`UNCLASSIFIED` → triage
E14	Tamil-transliterated text ("thanni varala")	Fuzzy/lexicon hit or triage — never a wrong confident domain

#	Case	Expected
E15	Resolve attempted with no docket	`GUARD_FAILED` 422
E16	Docket failing the configured pattern	`INVALID_DOCKET` 422
E17	Docket corrected after entry	`docket_no` updated; `audit_event` holds the old and new values
E18	Resolve → reopen → resolve again	Credit, reversal, new credit. Net +2. Three ledger rows
E19	Penalty backdated into a closed period	Applied to correct period, manager warned
E20	Attender A acts on attender B's task	403 `NOT_ASSIGNED`
E20a	Google sign-in by a non-allowlisted email	Rejected; no `app_user` row created
E20b	Sign-in by an allowlisted but `active = false` user	Rejected
E20c	Allowlisted email differing in case or surrounding space	Admitted — matching is normalised
E21	Month boundary at 02:00 IST on the 1st	Falls in the new month (NFR-7)
E22	Sheets API down for 6 hours	Backoff, degraded banner, full catch-up on recovery
E23	Anomaly detector below its `n` gate	Emits nothing (Sec. 8.3)
E24	`MAD = 0` in A1	Falls back to `3 × median`, no divide-by-zero
E25	Attender queue with 0 tasks	Empty state, not an error
E26	Leaderboard with an attender at 0 points	Appears, ranked last (`LEFT JOIN`, Sec. 7.8)
E27	Same browser re-subscribes to push	Existing row updated, not duplicated (`uq_push_endpoint`)
E28	Push service returns 410 Gone	Subscription revoked; no further attempts
E29	Attender with no push subscription is assigned a task	Assignment succeeds; push skipped silently
E30	Push service unreachable for an hour	Retries then `FAILED`; **no state transition affected**
E31	Notification listener throws	The originating transaction still commits (`AFTER_COMMIT`)

12.5 Concurrency suite

Direct tests of Sec. 7.1. Each runs against real Postgres, using a `CountDownLatch` so threads start together, and asserts on **final database state** rather than on response codes.

#	Test	Setup	Assertion
X1	Double resolve, same key	10 threads, same task, same `Idempotency-Key`	Exactly 1 ledger row, +2 total, all 10 get 200
X2	Double resolve, different keys	10 threads, same task, distinct keys	Exactly 1 ledger row — `RESOLVE:{task}:{att}` collides regardless (Sec. 7.3)
X3	Reassign vs resolve	Manager reassigns while attender resolves	Exactly one succeeds; loser gets 409; ledger consistent with the winner

#	Test	Setup	Assertion
X4	Shared duty, concurrent completion	4 attenders, shared task, simultaneous	4 ledger rows (one each), task `COMPLETED` exactly once
X5	Over-claim	4 attenders claim a `k=1` duty at once	1 assignment; 3 receive `NO_SLOTS_LEFT`
X6	Concurrent ingestion	3 poll runs of the same rows in parallel	No duplicate complaints; ShedLock serialises
X7	Leaderboard under write load	Read while 100 credits are written	Every read internally consistent; final total exact
X8	Reopen race	Manager reopens while attender re-resolves	Net points correct; no orphaned credit
X9	**Proposal performance fixture**	4 attenders × 25 tasks resolved concurrently	100 ledger rows, 200 points, zero lost updates, p95 < 500 ms

X2 is the sharpest test in the suite: it proves that correctness does **not** depend on the client sending a good idempotency key. Even a buggy client generating a fresh UUID per retry cannot double-credit, because the ledger key is derived from the work, not from the request.

12.6 Acceptance-criteria traceability

Each stakeholder-facing criterion from proposal Sec. 8, mapped to what proves it:

Criterion	Verified by	Milestone
Ingests automatically, no copy-paste, no duplicates	E2, E3, E4, E9, E10, E11, X6 + live sheet demo	Mid-demo
Categorizes ≥90% into Civil/Electrical/RO	Sec. 12.3 holdout evaluation	Mid-demo
Assigns to the primary floor attender	Dispatch unit tests, E5–E8	Mid-demo
Attender queue on mobile: docket entry + resolve, +2 credited	E15–E18, X1, X2 + device walkthrough	Mid-demo
Manager logs −1 penalties and special duties	Ledger tests, E19	Final
Real-time leaderboard	Sec. 7.8 tests, X7, SSE integration test	Final
One-click clean `.xlsx` export	POI tests, manual inspection	Final
Anomaly detection, bottlenecks, visualizations	Sec. 8 detector tests on F4, E23, E24	Final
Attenders alerted on their phone when work is assigned (G8)	E27–E31 + device walkthrough on a real Android phone	Final

12.7 What we are not testing

Stated to keep the plan honest:

- **Browser matrix** — Chrome on Android and desktop only; that is what these five people use.
- **Load beyond NFR-1.** X9 is the stated performance fixture. We do not test 1000 concurrent users, because there will never be five.
- **Google Sheets API itself** — mocked at the port boundary (Sec. 5.8). We test our reconciliation logic, not Google's service.
- **Browser push services themselves** (FCM, Mozilla autopush) — mocked at the HTTP boundary. We test our subscription handling, payload construction and failure responses, not Google's delivery.

- **iOS Safari push** — attenders are on Android (Sec. 10.1).
- **The IITM EU CMS app** — no integration exists to test (Sec. 1.8).
- **Penetration testing** — out of scope per Sec. 11.1.

13. Delivery Plan

13.1 Strategy: a working system first, capability layered on top

The organising principle is **vertical slices, not horizontal layers**. Each phase ends with something the stakeholder could actually use, not a tier that does nothing until the next tier arrives. Building ingestion, dispatch, ledger and analytics in parallel and integrating at the end is how a solo project arrives at the demo with four half-finished subsystems and nothing to show.

Two consequences shape everything below:

1. **Phase 1 is a walking skeleton.** It deliberately does the *least interesting* version of every step — CSV upload instead of the Sheets API, manual assignment instead of the heuristic — so that a complaint can travel the entire path in the deployed application within the first sprint. Every later phase then replaces one segment of a working pipeline rather than adding an unproven one.
2. **Phase 2 is the one that matters.** At the end of it the stakeholder's 30–45 minutes a day are gone. Everything after that improves a system that already solves the problem.

Priority is decided by one question: *does the Hostel Manager's daily workload drop without this?* If no, it waits.

13.2 Fixed dates

Milestone	Date	Requires
Design document	**11 Sep 2026**	This document
Mid-demo	**9 Oct 2026**	Phases 1–2 complete and deployed
Final submission	**6 Nov 2026**	Phases 3–4 complete, Phase 5 as time allows

Solo developer alongside coursework, planned capacity **~15 h/week** — roughly 130 hours total.

```
gantt
    title Phased delivery — 8 Sep to 6 Nov 2026
    dateFormat YYYY-MM-DD
    axisFormat %d %b

    section Foundation
    Design doc + repo scaffold        :done, p0, 2026-09-08, 4d
    P1 Walking skeleton               :p1, 2026-09-12, 10d

    section Solving the problem
    P2 Automate the daily work        :p2, 2026-09-22, 14d
    Demo polish + deploy hardening    :pd, 2026-10-06, 3d
    MID-DEMO                          :milestone, m1, 2026-10-09, 0d

    section Completing the system
    P3 Points, penalties, reporting   :p3, 2026-10-10, 11d
    P4 Oversight + notifications      :p4, 2026-10-21, 10d
    P5 Hardening, tests, sign-off     :p5, 2026-10-31, 7d
    FINAL                             :milestone, m2, 2026-11-06, 0d
```

13.3 Phase 1 — Walking skeleton · 12–21 Sep

Goal: one complaint travels the whole path, in the deployed app. Nothing clever.

#	Item	Why it is in Phase 1
1.1	Maven project, package layout (Sec. 2.3), ArchUnit rules	Structure is cheapest to establish before there is code
1.2	Flyway migrations: `app_user`, `floor`, `floor_assignment`, `complaint`, `task`, `task_assignment`, `point_ledger`, `audit_event`	The spine of the schema
1.3	Auth: **Google sign-in with an allowlist** for every user, provisioning screen, two seeded `MANAGER` rows	Nothing is usable without login. Google Cloud console setup (consent screen, two redirect URIs) happens here
1.4	**CSV upload** → complaint created	Proves ingestion *without* depending on Google API credentials arriving on time
1.5	Manager list view + **manual assign** to an attender	The heuristic can wait; the assignment concept cannot
1.6	Attender mobile queue → *Resolve* → **+2 ledger row**	Proves the ledger and the idempotency key end to end
1.7	Leaderboard (`SUM` over the ledger)	The stakeholder's second-biggest pain, and nearly free once the ledger exists
1.8	Deploy to Render + managed Postgres; CI green with Testcontainers	Deploying on day 10 rather than day 40 is where deployment surprises are cheap

Exit criterion: on the deployed URL, the manager uploads a CSV, assigns a complaint to Ravi; Ravi opens it on his phone, taps Resolve; the leaderboard shows +2. **That is a working system** — thin, but real.

Explicitly not in Phase 1: Google Sheets, categorization, the dispatch heuristic, dockets, SLA, push, analytics, Excel.

13.4 Phase 2 — Automate the daily work · 22 Sep – 5 Oct

Goal: the manager stops touching the spreadsheet. This is the phase that solves the problem.

#	Item	Replaces
2.1	`ComplaintSource` port; Google Sheets adapter with header mapping (Sec. 5.2)	1.4's CSV-only input (CSV stays as the fallback adapter)
2.2	`response_key` dedupe + single-mode scan (Sec. 5.3); scheduled poll + **Sync Now**	Manual copy-paste
2.3	Rule + fuzzy lexicon classifier with confidence gate (Sec. 5.5)	Manual sorting into domains
2.4	**Triage queue** — one-click domain + assign for low-confidence items	The safety net that makes 2.3 trustworthy
2.5	Auto-dispatch: Floor + Load scoring, tie-breaks, `score_snapshot` (Sec. 6)	1.5's manual assignment
2.6	Full state machine + transition table (Sec. 4), replacing ad-hoc status writes	Ad-hoc updates

#	Item	Replaces
2.7	Docket entry + the `mark_resolved` guard (T11, T14)	Verbal chasing of inspection IDs
2.8	Priority assignment, SLA due dates, aging job (Sec. 4.5)	Required for the mid-demo commitment
2.9	Gold set built and labelled; first accuracy + coverage measurement (Sec. 12.3)	The 90% acceptance criterion becomes measurable

Exit criterion — the important one: a complaint submitted to the Google Form appears categorized and assigned, on the right attender's phone, **with no human action in between**. The manager's daily 30–45 minutes is gone.

13.5 Phase 3 — Complete the points story · 10–20 Oct

Goal: the manual month-end tally disappears too.

#	Item
3.1	Penalty interface (−1) with mandatory notes; manual adjustments and reversals
3.2	Special duties: manager posts with custom points, attender claims and completes (Sec. 4.4)
3.3	Monthly period handling, `effective_date`, backdating into a closed period with a warning (Sec. 7.9)
3.4	Apache POI daily + weekly `.xlsx` export
3.5	Complaint detail screen: full audit timeline, score breakdown, docket history
3.6	**Assisted admin log** (`act-as`) — manager records progress during roll-call (Risk 3 Plan B)

Exit criterion: the manager can run a full month — points, penalties, duties, export — without opening a spreadsheet.

13.6 Phase 4 — Oversight and reach · 21–30 Oct

Goal: things the manager could never do before, and reaching attenders without them looking.

#	Item
4.1	**Web Push** to attender phones: VAPID, subscription endpoints, service worker, settings (Sec. 9.6)
4.2	SSE live dashboard: activity feed, live counters, live leaderboard (Sec. 9.5)
4.3	**Shared-task concurrency**: multi-assignee duties, per-assignee credit, OCC completion (Sec. 7.5)
4.4	Analytics: stage durations from the audit log, MTTR / median / p90, bottleneck identification
4.5	Anomaly detectors A1 (MAD outlier), A2 (chronic location), A3 (Poisson surge) with their `n` gates
4.6	Chart.js dashboard visualizations, each showing its sample size

Exit criterion: a P1 assignment buzzes the attender's phone within 30 s, and the dashboard shows real MTTR and anomaly alerts over accumulated data.

13.7 Phase 5 — Hardening and sign-off · 31 Oct – 6 Nov

#	Item
5.1	Full test pass; coverage gates; the complete concurrency suite (X1–X9)
5.2	Final gold-set evaluation on the holdout
5.3	Security review (Sec. 11), Gitleaks and Dependency-Check clean
5.4	Accessibility and mobile performance pass (Sec. 10.5–10.6)
5.5	Standby deployment runbook (Sec. 2.6) and a rehearsed failover
5.6	**Stakeholder UAT** against Sec. 12.6; README, runbook, viva notes
5.7	**Sign-off**

13.8 Backlog — deliberately not scheduled

Genuinely optional. Each is additive, and none is required by an acceptance criterion. They are listed so the decision to skip them is visible rather than silent.

Item	Condition for building it
Amendment tracking for edited Form responses (Sec. 5.3)	Only if the Form has "edit response" enabled. Schema already reserves the `AMENDED` state
Offline replay queue (IndexedDB + service worker)	If attenders report losing actions in poor signal. Push already needs the service worker; this is the replay half only
OpenNLP second stage (Sec. 5.7)	Once enough triage corrections have accumulated to train on
Detectors **A4** (SLA cluster) and **A5** (workload imbalance)	Both need ≥4 weeks of history to emit anything, so neither can be demonstrated before 6 Nov anyway
Manager-editable lexicon UI	If the manager wants to tune categorization without a redeploy. Seeded by migration until then
Deletion detection (`VANISHED`)	Only if the manager reports deleting rows and wanting to know

13.9 Changes from the original proposal

Change	Reason
Delivery reorganised into vertical phases	A solo project integrating four subsystems at the end is the main schedule risk; a thin working system by 21 Sep removes it

Change	Reason
One PostgreSQL everywhere; SQLite and H2 dropped	SQLite cannot demonstrate the concurrency control; the H2 standby left work with no path back to the cloud database (ADR-001)
Auth added as explicit Phase 1 scope	Absent from the proposal, but the schema, role matrix and deployment all depend on it
Rules-first classification; OpenNLP to backlog	Training-data availability is unproven; rules need none and are explainable (ADR-004)
SSE over WebSocket, without replay machinery	One-directional need; a refetch on reconnect gives the same screen (ADR-003)
Push notifications to attenders added	Stakeholder requirement after the proposal; reuses the service worker (ADR-011)
Eight mechanisms simplified or removed	Sized for a system with <10 users, and for a viva where every line must be explainable (ADR-012)

13.10 Risk and the cut line

There is **no slack**; a lost week comes out of scope, not quality. Because the phases are ordered by value, the cut line is simply *how far down Sec. 13.6 and Sec. 13.7 we get* — and the system remains useful at every point, which is the whole reason for slicing it this way.

Risk	Trigger	Response
Sheets API credentials not granted in time	Nothing by 22 Sep	Phase 1 already ships on CSV; build 2.1 against a personal test sheet
Real complaint data unavailable for the gold set	Nothing by 22 Sep	Use a synthetic corpus and state that caveat with the accuracy figure
Categorization misses 90%	Holdout below target at Phase 2 exit	Expand the lexicon from observed misses; raise τ, trading coverage for accuracy since accuracy is the stated criterion
Free-tier deploy proves unusable	Failure during Phase 1	Found on day 10 rather than day 40 — that is why 1.8 is in Phase 1
Coursework crunch	Any phase slips >3 days	Drop from the bottom of Phase 4, then Phase 5.5. Never cut Phases 1–3

Never cut, because each is load-bearing for a stakeholder acceptance criterion or a core integrity guarantee: ingestion idempotency, the ledger and its constraints, the state machine, dispatch, the attender mobile flow, penalties, the leaderboard, and the daily Excel export.

14. Decision Record

Architecture Decision Records for the choices that shaped this design. Each states what was decided, what it cost, and what would make us revisit it.

ADR-001 · One PostgreSQL database; the standby runs against it too

Status: Accepted · 8 Sep 2026 · **Revised** 8 Sep 2026 after clarifying what "outage" means

Context. The proposal specified SQLite or embedded H2 for local development with PostgreSQL in production, plus a local fallback server for outages. The original decision kept H2 for the fallback and constrained the schema to the SQL subset both engines accept.

The stakeholder then clarified that **the outage being designed for is the cloud platform being unavailable, not the internet.** Office internet is reliable.

Decision. PostgreSQL everywhere and only: development (Docker Compose), test (Testcontainers), production, and the standby. The standby is **the same jar with the same `DATABASE_URL`**, started on the manager's desktop when Render or Railway is down. H2 and SQLite are dropped entirely.

Rationale. - SQLite serialises writers, so the shared-task concurrency control (Sec. 7) would pass its tests for the wrong reason and prove nothing about production. - H2 and Postgres diverge on constraint-violation behaviour, isolation semantics and locking — exactly the surface our correctness argument (Sec. 7.3) rests on. - **Decisively:** an H2 standby seeded from a nightly dump left work done during an outage with **no path back into the cloud database**. Two diverging copies of the ledger is the precise failure this system exists to prevent. Pointing both jars at one database does not solve that problem — it means it cannot occur. - It also deletes the portable-SQL constraint from every future migration, a second dialect to test, and a Spring profile.

Consequences. Requires Docker locally for development. Attenders cannot reach the standby (it is on `localhost`), so during a platform outage the manager works alone — acceptable, since that is strictly better than today's all-manual process. The residual risk is the managed database itself failing, which no application design fixes; nightly dumps bound the loss and the manager falls back to pen and paper for those hours, as they do today.

Revisit if: office internet stops being reliable, or the managed database's availability proves inadequate.

ADR-002 · Google sign-in for everyone, admitted by an allowlist

Status: Accepted · 9 Sep 2026 — third revision; supersedes both local credentials and the manager/attender split

Context. One manager and ~4 attenders. Only a pre-registered set may have access, and people without an institute account must be usable.

The decision trail matters more than the outcome, so all three revisions are kept:

1. *Local credentials for everyone*, justified primarily by "OAuth cannot complete a redirect flow without internet, so it would fail on the offline fallback."
2. That argument was **withdrawn as factually wrong** once the stakeholder clarified the outage is the cloud platform, not the network. Re-justified on account recovery, which was then found to be **overstated** — a manager can repoint the allowlist. Decision moved to Google for attenders, local password retained for the manager.
3. *This revision:* the three reasons for keeping the manager local do not survive scrutiny either.

Decision. Every user — manager and attenders — authenticates with **Google OIDC**, admitted by an allowlist against `app_user.email`. No passwords, no PINs, no remember-me. Two `MANAGER` rows are provisioned as break-glass.

Why the manager's local password was dropped. - *"Needs the standby during an outage."* Google permits `http://localhost` redirect URIs. Register the standby URI alongside the cloud one and it signs in normally. - *"A demo must work regardless of network."* The application is hosted on Render. If the network fails the demo fails anyway, so

a local password buys nothing in its own motivating scenario. - *"They are the support desk."* Genuine — and answered by a **row rather than a mechanism**: two provisioned `MANAGER` accounts, plus the developer's database access as ultimate recovery. A forgotten password would have ended in exactly the same place, with one more mechanism built.

Rationale. - **One mechanism, one sentence to defend:** every user authenticates with Google and is admitted by the allowlist. That is a better answer under questioning than a two-path scheme with four justifications. - **No credential exists in the schema at all.** A database breach yields nothing to steal. - Deletes `password_hash`, `username`, the BCrypt encoder, the login form and controller, password rate limiting, and one of two `SecurityFilterChain` branches. - IITM runs Google Workspace, so an institutional account is recoverable through IT — more reliable than a password a user may forget. The allowlist is domain-agnostic, so a personal Gmail works identically.

Consequences. - Sign-in depends on Google availability. Since the app is cloud-hosted, a network failure already takes the system down independently of authentication. - **Authentication is not integration-tested end to end.** The authorization-code flow is Spring Security's code; the allowlist decision is ours and is unit-tested directly, with `@WithMockUser` elsewhere (Sec. 12.1). Recorded rather than glossed. - An OAuth client secret and two redirect URIs to maintain (Sec. 11.5). - Email addresses become stored personal data (Sec. 11.4). - Recovery is partly external: a locked-out user needs *some* working Google account first.

Revisit if: users turn out not to control their own Google accounts in practice, or Google sign-in proves unreliable on the devices actually in use.

ADR-003 · SSE, not WebSocket, for live updates

Status: Accepted · 8 Sep 2026

Context. The proposal listed "Spring WebSocket / SSE" for real-time manager notifications.

Decision. Server-Sent Events, with a transactional outbox for replay.

Decision (revised). Server-Sent Events, published on `AFTER_COMMIT`, with **no replay machinery** — a client that reconnects refetches current state.

Rationale. The requirement is strictly one-directional: the server pushes duty updates to one manager dashboard. SSE is plain HTTP, traverses PaaS proxies cleanly, and reconnects automatically with no STOMP broker or fallback negotiation. Treating it as a *latency optimisation over polling* rather than a delivery guarantee is what lets the outbox and `Last-Event-ID` replay go: a refetch after reconnect renders the identical screen (ADR-012).

Consequences. No client→server push channel; state changes go over normal HTTP POSTs, which is what we wanted anyway since they must pass through the state machine. An event emitted while the dashboard is disconnected is lost — harmless, because the refetch on reconnect shows the resulting state. SSE is limited to ~6 concurrent connections per browser origin, irrelevant at one manager.

Revisit if: bidirectional realtime is ever needed (e.g. live attender-to-manager chat).

ADR-004 · Rule-based lexicon as the primary classifier; OpenNLP deferred

Status: Accepted · 8 Sep 2026

Context. The proposal names Apache OpenNLP for "rule-assisted text categorization", with an acceptance criterion of ≥90% accuracy.

Decision. A weighted lexicon with fuzzy matching and a confidence gate is the primary and only required classifier. OpenNLP `DocumentCategorizer` is designed in as an optional second stage, consulted only for items the rules route to triage, and is tagged Extended (Sec. 13.3).

Rationale. - A supervised model needs a few hundred labelled examples per class. We have no confirmation that volume of labelled data exists, and the project cannot depend on an unverified input. - Complaint text is short, misspelled and often code-mixed Tamil/English transliteration — where lexicon + edit-distance degrades gracefully and a thin-data

model does not. - The lexicon is editable by the manager without a redeploy. - Every decision is explainable by naming the terms that fired — which matters for the viva and for the manager's trust in the triage card (Sec. 10.3).

Consequences. Requires curating a domain lexicon (a few hours, with the stakeholder). Ceiling on accuracy is lower than a well-trained model's — mitigated by the confidence gate routing uncertain items to triage rather than guessing. Production triage corrections accumulate the labelled corpus that would make stage two viable later, so nothing is foreclosed.

Revisit if: holdout accuracy stalls below 90% with coverage above 70%.

ADR-005 · Greedy per-complaint dispatch, not global optimisation

Status: Accepted · 8 Sep 2026

Context. Assignment could be formulated as a global optimisation (Hungarian algorithm, min-cost flow) over a batch of complaints and attenders.

Decision. Greedy `argmax` of a two-term weighted score (floor proximity, workload), per complaint, with a deterministic tie-break chain (Sec. 6.2).

Rationale. Complaints arrive one at a time from a 3-minute poll; a typical batch is 1–3 items, and the optimal assignment of a 1-item batch *is* the greedy one. With $|A| = 4$ the theoretical gain is negligible, while batch optimisation would require holding complaints back to accumulate a batch — adding latency to exactly the P1 cases that matter most. Greedy is also explainable, and "Ravi 0.78 vs Kumar 0.50" is something a manager can challenge.

Consequences. No global optimality guarantee. Anti-starvation must be handled explicitly, which it is, via the tie-break chain and the load term.

Revisit if: attender count grows past ~15, or complaints start arriving in large batches.

ADR-006 · Pessimistic locking for duty claims, optimistic elsewhere

Status: Accepted · 8 Sep 2026

Context. Claiming a special-duty slot is a read-modify-write on a count (`active < k`). The house style everywhere else is optimistic locking.

Decision. `SELECT ... FOR UPDATE` on the task row for claims only.

Rationale. Optimistic locking would work correctly, but its user-visible failure is a retry prompt for someone standing in a corridor. Contention here is trivially low — 4 people, a handful of duties a week — so a lock held for microseconds costs nothing and lets the loser get an immediate, unambiguous "already claimed". The clarity is worth the inconsistency in style, and the inconsistency is documented rather than accidental.

Consequences. One deliberate deviation from the OCC default. `uq_task_attender` provides a second, independent guarantee against self-double-claim.

Revisit if: claim contention ever becomes non-trivial.

ADR-007 · `READ COMMITTED` isolation, not `SERIALIZABLE`

Status: Accepted · 8 Sep 2026

Context. The proposal calls for "strict transactional isolation". The strongest reading is `SERIALIZABLE`.

Decision. PostgreSQL's default `READ COMMITTED`, with correctness carried by uniqueness constraints and version checks.

Rationale. Every invariant we depend on — no double-credit, no lost update, no over-claim — is enforced by a `UNIQUE` constraint, an `@Version` check, or a row lock. All three hold at `READ COMMITTED`. `SERIALIZABLE` would add serialization failures and retry complexity across every transaction to buy a guarantee already obtained more cheaply and more locally.

Consequences. Non-repeatable reads are possible within a transaction; no invariant relies on their absence. Any *future* invariant depending on a multi-row consistent snapshot would need explicit locking or a stronger level for that transaction alone.

Revisit if: a new requirement needs cross-row consistency that constraints cannot express.

ADR-008 · Append-only ledger; no stored point balance

Status: Accepted · 8 Sep 2026

Context. The obvious implementation of a point total is a counter column on the attender.

Decision. `point_ledger` is append-only. Balances are `SUM` aggregates. Corrections are compensating rows, never updates or deletes. No `attender.points` column exists.

Rationale. - A counter has a lost-update race on every increment; an insert with a unique key does not. - Bonus disputes require answering "why is this number 34?" — a ledger answers line by line, a counter cannot answer at all. - Reversal-by-compensation keeps the reopen cycle (T17) auditable. - At NFR-1 volume the aggregate is an index scan over a few hundred rows; there is no performance argument for denormalising.

Consequences. Every point read is an aggregate query. Accepted, and measured: ~600 rows per period. If volume ever justified it, the right answer is a materialized view refreshed on commit — never a mutable counter, which would give back the property that makes this design correct.

Revisit if: ledger volume exceeds ~1M rows.

ADR-009 · Content-derived identity for source rows, not row index

Status: Accepted · 8 Sep 2026

Context. Ingestion must detect new, edited and deleted Google Form responses without a change feed.

Decision. Identity is `response_key = SHA256(timestamp || email)`; change detection is a separate `content_hash` over all cells. Row index is stored as a positional hint only.

Rationale. Deleting one sheet row shifts every index below it. Keying on `(sheet, row_index)` would report the entire tail as amended after a single deletion, and — depending on the reconciliation rule — could duplicate hundreds of complaints in one poll. A Google Form response retains its submission timestamp and submitter across edits, so a content-derived key is stable across both row shifts and edits. Separating identity from change detection is what lets one algorithm handle inserts, amendments and deletions coherently.

Consequences. Requires the Form to collect email, or timestamp-only keys with an in-batch disambiguator for same-second submissions (Sec. 5.3). Deletion can only be concluded from a **full** scan, never an incremental one — the most important guard in the algorithm.

Revisit if: the Form is replaced by a source with stable native row IDs.

ADR-010 · Modular monolith, not microservices

Status: Accepted · 8 Sep 2026

Context. The proposal describes several distinct subsystems (ingestion, dispatch, ledger, analytics) that could be separate services.

Decision. One Spring Boot jar with enforced internal module boundaries (Sec. 2.3–2.4), ports and adapters only where an external system is involved.

Rationale. At ≤50 complaints/day with 4 users, distribution would add network failure modes, deployment complexity and eventual-consistency problems in exchange for scale we will never need. Critically, our core correctness guarantees (Sec. 7) rely on **single-database transactions**; splitting the ledger from the workflow would turn a uniqueness constraint into a distributed-transaction problem. Module boundaries are drawn so a component *could* be extracted if that ever changed.

Consequences. One deployment unit, one database, one failure domain. ArchUnit enforces the boundaries that would otherwise erode (Sec. 2.4).

Revisit if: the system is ever extended to multiple hostels with independent operations.

ADR-011 · Web Push with VAPID for attender notifications, not FCM or a native app

Status: Accepted · 8 Sep 2026

Context. Attenders must be alerted on their phone when work is assigned, without having to open the app and look (G8). This was added after the proposal, which had listed manager-side SSE only.

Decision. The **Web Push protocol with VAPID**, delivered to the service worker the PWA already registers for offline replay. No Firebase project, no Google Play binary, no third-party push SDK. Library: `nl.martijndwars:web-push`.

Rationale. - The service worker is **already required** for the offline action queue (Sec. 10.2). Push reuses that registration, so the marginal cost is a subscription table, an endpoint pair and a message handler — not a new client platform. - VAPID needs only a self-generated key pair. FCM would mean a Firebase project, a Google account dependency and an SDK, to reach the same Android Chrome that speaks Web Push natively. - A native app would need Play Store distribution, signing keys and an update path for four people who already use the web app — plainly disproportionate. - Payloads are end-to-end encrypted, so the relaying push service sees only ciphertext.

Consequences. - Attenders must grant browser notification permission, and a denial is not recoverable from inside the app. Mitigation: ask only *after* first successful login, once the app has shown its value (Sec. 10.2), and expose a settings toggle so people dial alerts down rather than revoking them. - Push works from the standby deployment too, since internet is available and the standby talks to the same database (Sec. 2.6). The only loss during a platform outage is that attenders cannot reach the app on `localhost` to act on an alert. - iOS would require an installed PWA on Safari 16.4+. Out of scope; attenders are on Android. - Delivery is best-effort and no state transition depends on it (NFR-10). The in-app queue stays the source of truth, so a missed push degrades convenience, never correctness.

Revisit if: attenders move to iOS in numbers, or delivery reliability proves inadequate in the field — at which point FCM's data channel would be the next step, not a native app.

ADR-012 · Deliberate simplification pass; ship a working system first

Status: Accepted · 8 Sep 2026

Context. The design as first written was sized for a production system with many users. The real system has **fewer than ten users**, one hostel, and one hard constraint that outweighs elegance: every line must be explainable in a viva. A review against that constraint found several mechanisms that added implementation complexity without adding user-visible capability.

Decision. Remove them, and reorganise delivery so a **working end-to-end system exists early** (Sec. 13) with optional capability layered on afterwards rather than everything built in parallel.

Removed	Replaced by	Why it was not carrying its weight
Skill + Continuity dispatch terms	Floor + Load only	Skill was a constant `0.5` for every candidate, so it *mathematically could not* change the ranking. Continuity cost a per-candidate lookback query for a 10% nudge
Transactional outbox + `Last-Event-ID` replay	Refetch current state on reconnect	Identical screen for one manager, minus a table, a poller and replay bookkeeping
`push_delivery` table	`fail_count` + a log line	A per-attempt audit table for four phones
Hand-rolled device tokens, then the remember-me table itself	Nothing — Google re-auth is one tap	Superseded by ADR-002; a persistent-token scheme saved an interaction that now costs nothing
`cms_docket` table + 5-state docket machine	Three columns on `complaint`	The history duplicated `audit_event`; the `VERIFIED` state gated nothing by its own definition
`VANISHED` state + nightly full-sheet scan	Ignore deletions	Bought one notification, cost a second scan mode and a review path
Temporal `floor_assignment`	Current coverage only	Date-filtered every dispatch query to serve rotation that does not happen; `score_snapshot` already explains past decisions
H2 standby database	Same managed Postgres (ADR-001)	Removed the portable-SQL constraint *and* an unsolved data-divergence problem

Deferred rather than deleted: amendment tracking for edited Form responses (Sec. 5.3) is a Phase 5 feature. It only applies if the Form enables response editing, and the schema already reserves the `AMENDED` state, so adding it later is additive.

What was explicitly kept, because it is the difference between this and a CRUD app: the append-only ledger with work-derived idempotency keys, the state machine and its transition table, content-derived ingestion identity, optimistic locking, and confidence-gated classification with a triage fallback. Every one of these prevents a specific, nameable failure.

Consequences. Roughly a third less code to write, test and explain. Two schema decisions (dockets, coverage) trade a small amount of future flexibility for present clarity; both are additive to reverse. The guiding rule going forward: **prefer more capability over more mechanism**, and where a framework already solves something, use its solution.

Revisit if: user count grows past ~20, or a removed mechanism turns out to be load-bearing for a requirement we have not anticipated.